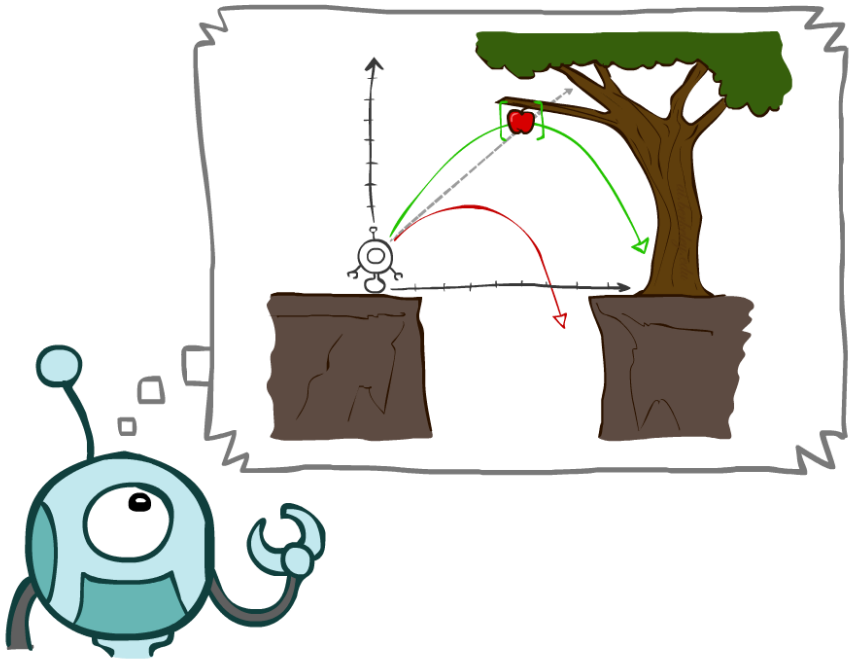
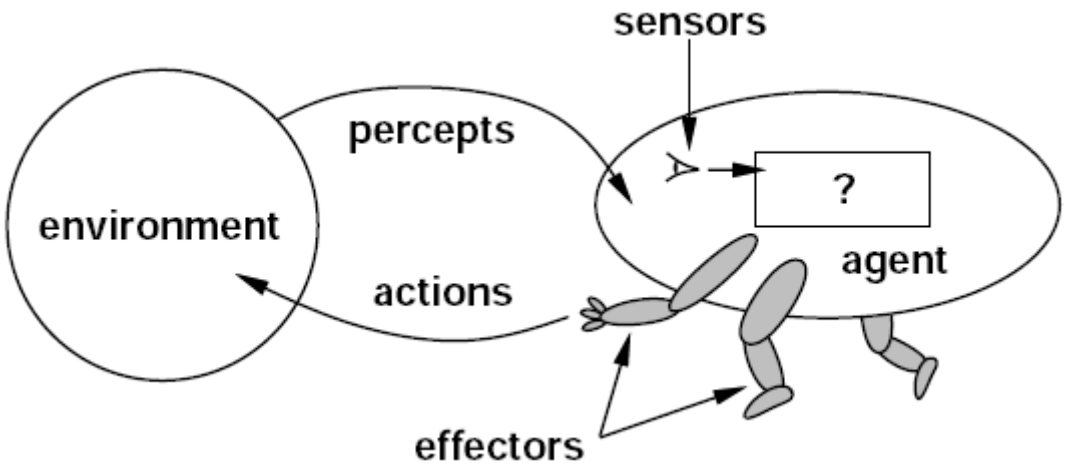
Intelligent Agents



Instructor: Husnain Haider
Slide credits: AI BERKELEY

What is an Agent?

- *"Intelligent agents continuously perform three functions: **perception** of dynamic conditions in the environment; **action** to affect conditions in the environment; and **reasoning** to interpret perceptions, solve problems, draw inferences, and determine actions."*



Intelligent Agents

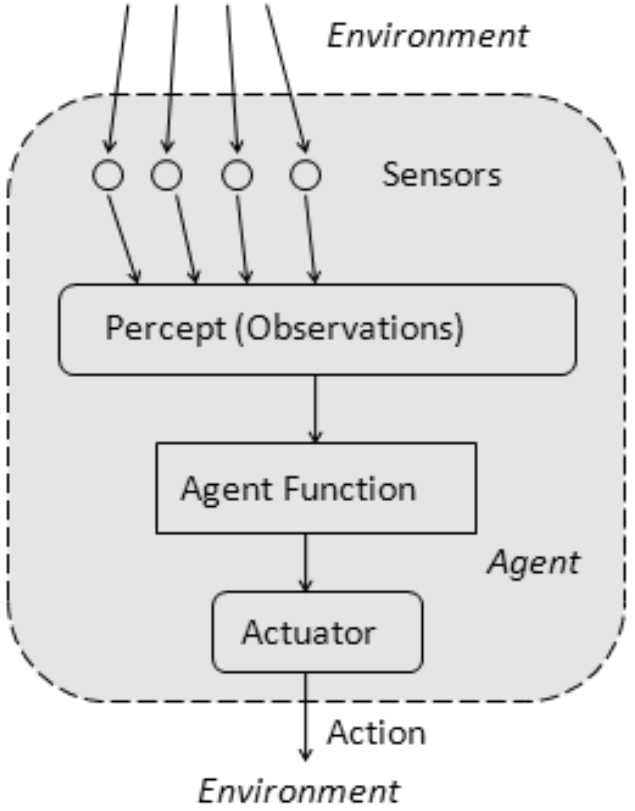
- An intelligent agent may learn from the environment to achieve their goals.
- Following are the main four rules for an AI agent:
  - **Rule 1:** An AI agent must have the ability to perceive the environment.
  - **Rule 2:** The observation must be used to make decisions.
  - **Rule 3:** Decision should result in an action.
  - **Rule 4:** The action taken by an AI agent must be a rational action.

Example of Agents

- Human Agent
  - eyes, ears, skin, taste buds, etc. for sensors
  - hands, fingers, legs, mouth, etc. for actuators
- Robot
  - camera, infrared, bumper, etc. for sensors
  - grippers, wheels, lights, speakers, etc. for actuators
- Software Agent
  - functions as sensors
    - information provided as input to functions in the form of encoded bit strings or symbols
  - functions as actuators
    - results deliver the output

Agent Function and Program

- **Behavior of Agent** – It is the action that agent performs after any given sequence of percepts
- Agent's behavior is *mathematically* described by
  - **Agent function**
  - A function mapping any given percept sequence to an action
- *Practically* it is described by
  - An **agent program**
  - The real implementation
- Problems:
  - what is " the right thing"
  - how do you measure the "best outcome"



Specifying the Task Environment

Performance of Agents

Performance of Agents

- Behavior and performance of IA in terms of agent program.
  - Perception to Action Mapping
  - Ideal mapping: specifies which actions an agent ought to take at any point in time.
- Performance measure: a subjective measure to characterize how successful an agent is (e.g., speed, power, accuracy, etc.)
- **PEAS:**
  - **Performance measure,**
  - **Environment,**
  - **Actuators,**
  - **Sensors**

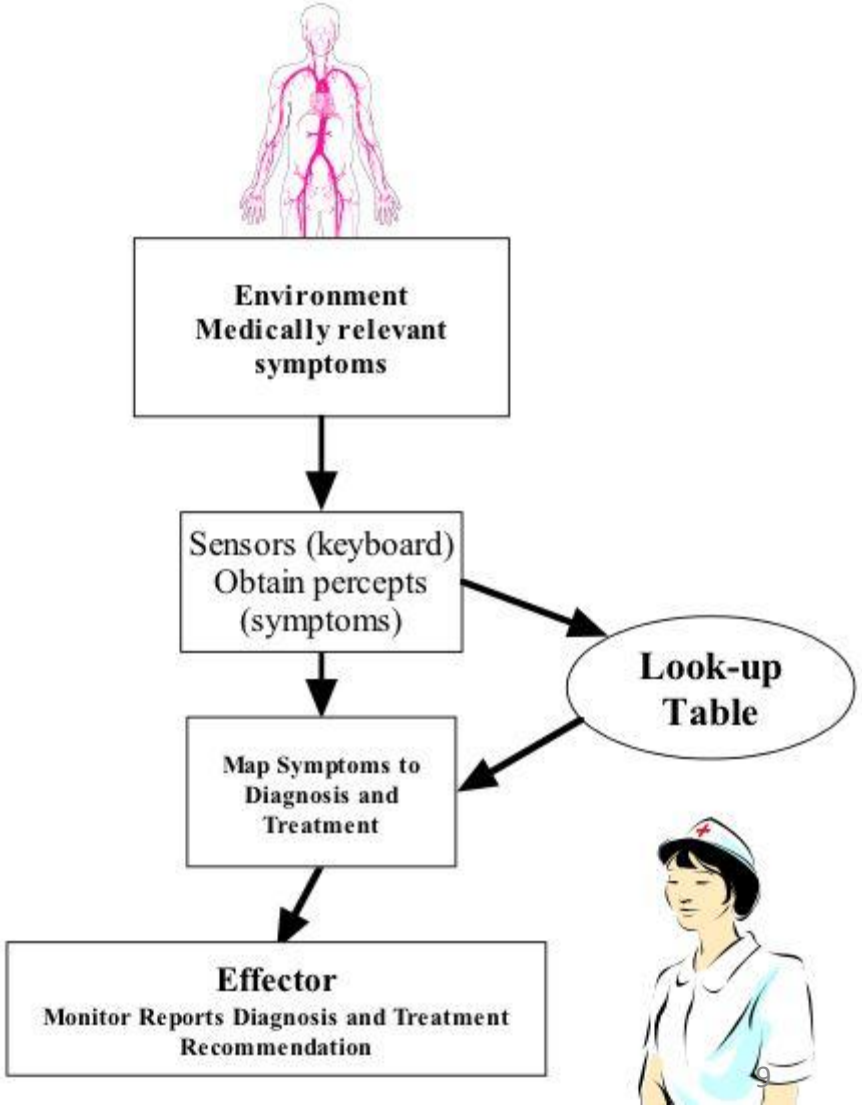
PEAS: Automated Taxi

- Performance measure
  - Correct Destination, Fuel, time, cost, violations, safety, comfort, profit
- Environment
  - rural, urban, other drivers, customers
- Actuators
  - Steering, brake, fuel, display/speaker
- Sensors
  - Camera, radar, accelerometer, engine sensors, microphone



PEAS: Medical diagnosis system

- Performance measure:
  - Healthy patient, minimize costs, lawsuits
- Environment:
  - Patient, hospital, staff
- Actuators:
  - Screen display (questions, tests, diagnoses, treatments, referrals)
- Sensors:
  - Keyboard (entry of symptoms, findings, patient's answers)



PEAS: Part-picking robot

- Performance measure:
  - Percentage of parts in correct bins
- Environment:
  - Conveyor belt with parts, bins
- Actuators:
  - Jointed arm and hand
- Sensors:
  - Camera, joint angle sensors





PEAS: Interactive English tutor

- Performance measure:
  - Maximize student's score on test
- Environment:
  - Set of students
- Actuators:
  - Screen display (exercises, suggestions, corrections)
- Sensors:
  - Keyboard

Environment Types

Properties of environments


vs.


- **Fully observable** vs. **partially observable**
- **Or Accessible vs. inaccessible**
  - Can the agent observe the complete state of the environment?
  - If an agent's sensors gives it access to the complete state of the environment, then we say that environment is fully observable to the agent.
  - A fully observable environment is convenient because the agent need not maintain any internal state to keep track of the world.
  - ***Is the current state of the world fully known at each step?***

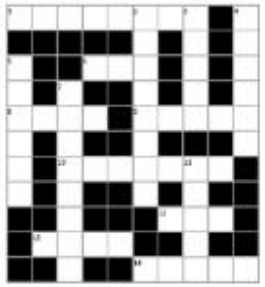
Properties of environments



- **Deterministic** vs. **nondeterministic (Stochastic).**
  - Is there uncertainty in how the world works?
  - If the next state of the environment is completely determined by the current state and the actions selected by the agents, then we say the environment is deterministic.
  - If the environment is inaccessible, then it may *appear* to be nondeterministic (bunch of uncertainties).

  - Taxi driving is clearly stochastic in this sense, because one can never predict the behavior of traffic exactly.

Environment Types



vs.



- **Episodic** vs. **Sequential**:
  - Episodic: next episode doesn’t depend on previous actions.
  - The agent's experience is divided into atomic "episodes"
  - each episode consists of the agent perceiving and then performing a single action, and
  - the choice of action in each episode depends only on the episode itself.
  - the next episode does not depend on the actions taken in previous episodes
  - For example, an agent that has to spot defective parts on an assembly line (EPISODIC)
  - Chess and Taxi driver (SEQUENTIAL).

Environment Types

- **Static** vs. **Dynamic**:
  - If the environment can change while an agent is agent is performing action, thenwe say the environment is dynamic for that agent; otherwise, it is static.
  - Dynamic environments are continuously asking the agent what it wants to do
  - Static environments are easy to deal with, because the agent does not keep on looking at the environment while it is deciding on an action.
  - The environment is **semi-dynamic** if the environment itself does not change with the passage of time but the agent's performance score does.

Environment Types

- **Discrete** vs. **continuous**:
  - Is there a finite (or countable) number of possible environment states?
  - The environment is discrete if the number of actions and possible states of the environment is finite otherwise it is continuous.
  - If there are a limited number of distinct, clearly defined percepts and actions, we say that the environment is discrete.
    - Chess, since there are a fixed number of possible moves on each turn.
    - Taxi driving is continuous.

Types of Environment

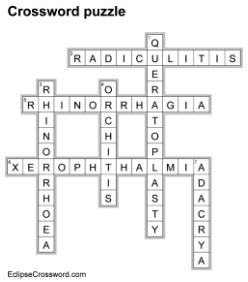
- **Single Agent** vs. **Multi Agent**
  - Is the agent the only thing acting in the world?
  - In the single agent environment there is only one agent
    - A computer software playing crossword puzzle
  - In multiagent systems, there are more than one active agents
    - Video games

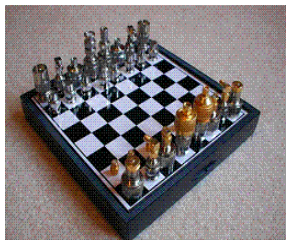


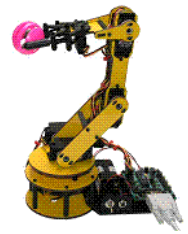
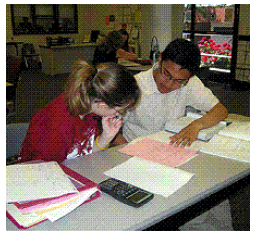
vs.



Environment Types Example





	Crossword puzzle	Chess	Taxi driving	Part-picking robot	English tutor
Observable	Yes	Yes	No	No	No
Deterministic	Yes	Yes	No	No	No
Episodic	No	No	No	Yes	No
Static	Yes	Yes	No	No	No
Discrete	Yes	Yes	No	No	Yes
Agents	Single	Multi	Multi	Single	Multi

Structure of Agents

Agent design

- **The environment type largely determines the agent design**
  - ***Fully/partially observable*** => agent requires ***memory*** (internal state)
  - ***Discrete/continuous*** => agent may not be able to enumerate ***all states***
  - ***Stochastic/deterministic*** => agent may have to prepare for ***contingencies***
  - ***Single-agent/multi-agent*** => agent may need to behave ***randomly***

Skeleton Agent Program

- Basic framework for an agent program

```
function SKELETON-AGENT(percept) returns action
  static: memory

  memory   := UPDATE-MEMORY(memory, percept)
  action   := CHOOSE-BEST-ACTION(memory)
  memory   := UPDATE-MEMORY(memory, action)

return action
```

Table Agent Program

- agent program based on table lookup

```
function TABLE-DRIVEN-AGENT(percept) returns action
  static: percepts  // initially empty sequence*
          table     // indexed by percept sequences
                    // initially fully specified

  append percept to the end of percepts
  action      := LOOKUP(percepts, table)

return action
```

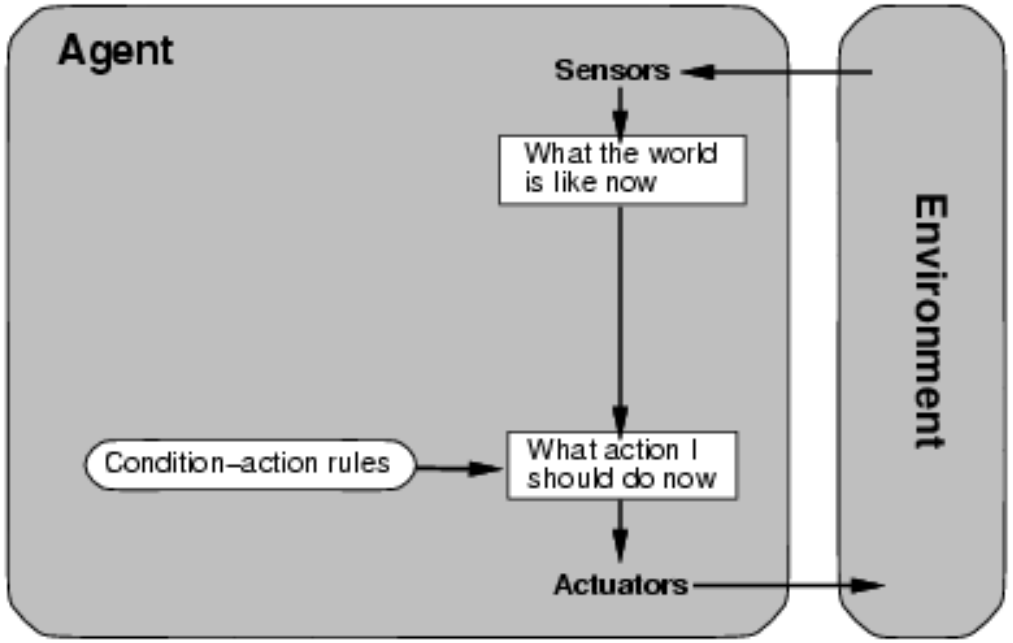
* Note: the storage of percepts requires writeable memory

Structure of Agent

- Different ways of achieving the mapping from percepts to actions
- Types of agents (increasing in generality and ability to handle complex environments)
  - Simple reflex agents
  - Model-based reflex agents
    - keep track of the world
  - Goal-based agents
    - work towards a goal
  - Utility-based agents
  - Learning agent

Simple reflex agents

- Act only on the basis of the **current percept, ignoring the rest of the percept history**.
- The agent function is based on the ***condition-action rule***: if condition then action.
- the vacuum agent is a simple reflex agent, because its decision is based only on the current location and on whether that location contains dirt.
- Fully-observable

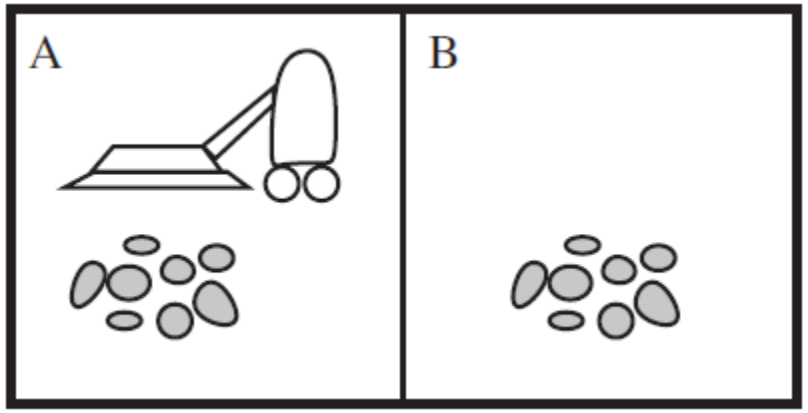


Simple Reflex Agents

- The Simple reflex agents are the simplest agents. These agents take decisions on the basis of the current percepts and ignore the rest of the percept history.
- These agents only succeed in the fully observable environment.
- The Simple reflex agent works on Condition-action rule, which means it maps the current state to action.
- Problems for the simple reflex agent design approach:
  - They have very limited intelligence
  - Mostly too big to generate and to store.
  - Not adaptive to changes in the environment.

Example: Vacuum Agent

- **Performance**?
  - 1 point for each square cleaned in time T?
  - #clean squares per time step - #moves per time step?
- **Environment**: vacuum, dirt, multiple areas defined by square regions
- **Actions**: left, right, suck, idle
- **Sensors**: location and contents
  - [A, dirty]



If status=Dirty then return Suck
else if location=A then return Right
else if location=B then right Left

Simple Reflex Agent

function SIMPLE-REFLEX-AGENT(*percept*) **returns** an action
  persistent: *rules*, a set of condition–action rules

  state ← INTERPRET-INPUT(*percept*)
  rule ← RULE-MATCH(*state*, *rules*)
  action ← *rule*.ACTION
  return *action*

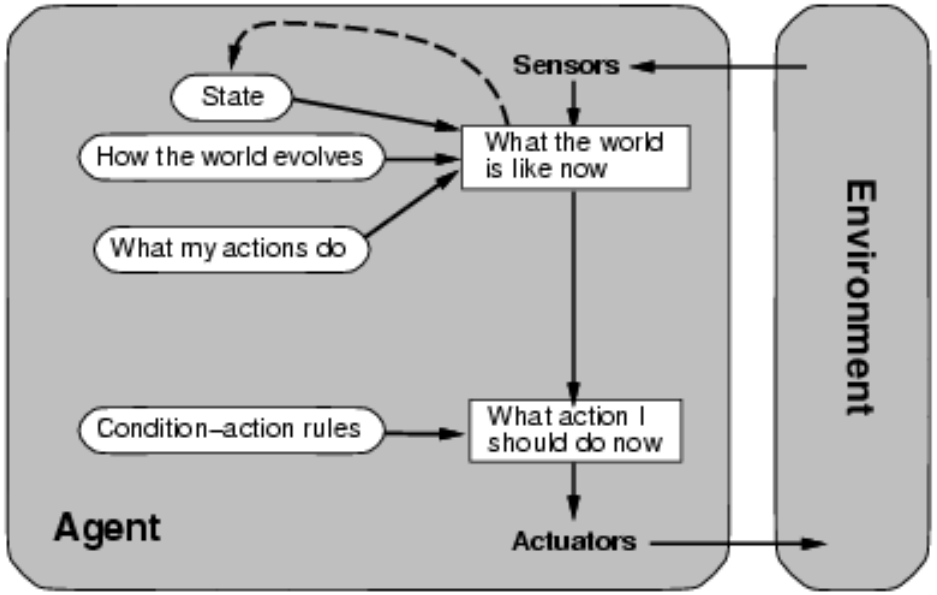
Figure 2.10 A simple reflex agent. It acts according to a rule whose condition matches the current state, as defined by the percept.

```
def simple_reflex_agent(percept):
        state = get_state_from_percept(percept)
        rule = match_rule(state, rules)
        action = rule.Action
        return action
```

simple_reflex_agent.py hosted with ❤ by GitHub view raw

Model-based reflex agents

- Keep track of the part of the world it can’t see now.
- Its current state is stored inside the agent ---- **Internal state**
- Percept history and impact of action on the environment can be determined by using internal model.
- It then chooses an action in the same way as reflex agent.
- Partially-observable

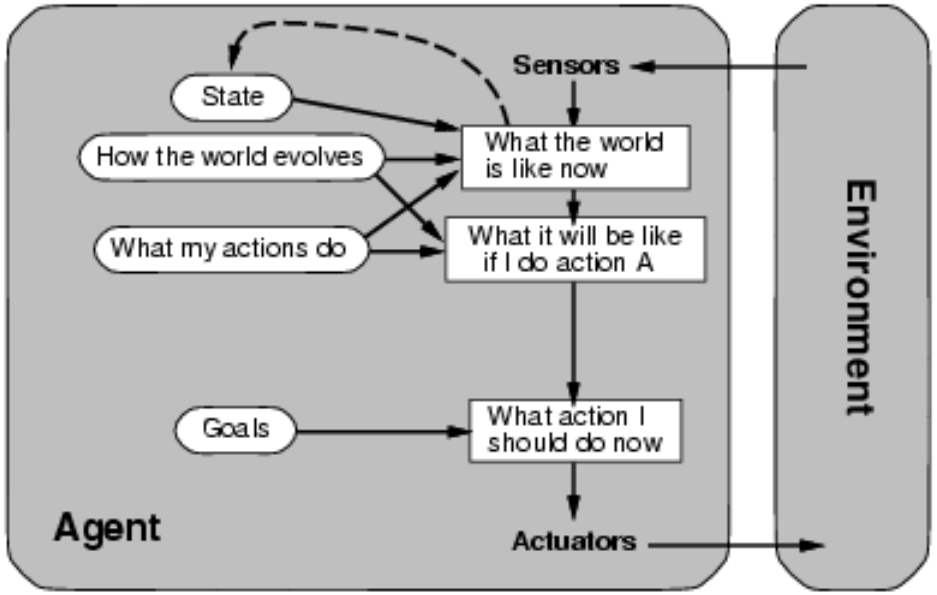


Model-Based Reflex Agents

- The Model-based agent can work in a partially observable environment, and track the situation.
- A model-based agent has two important factors:
  - **Model:** It is knowledge about "how things happen in the world," so it is called a Model-based agent.
  - **Internal State:** It is a representation of the current state based on percept history.
- These agents have the model, "which is knowledge of the world" and based on the model they perform actions.
- Updating the agent state requires information about:
  - How the world evolves
  - How the agent's action affects the world.

Goal-based agents

- Expand on the capabilities of the model-based agents, by using "goal" information.
- Goal information describes situations that are desirable.
- This allows the agent a way to choose among multiple possibilities, selecting the one which reaches a goal state.
- Search and planning



Goal Based Agents

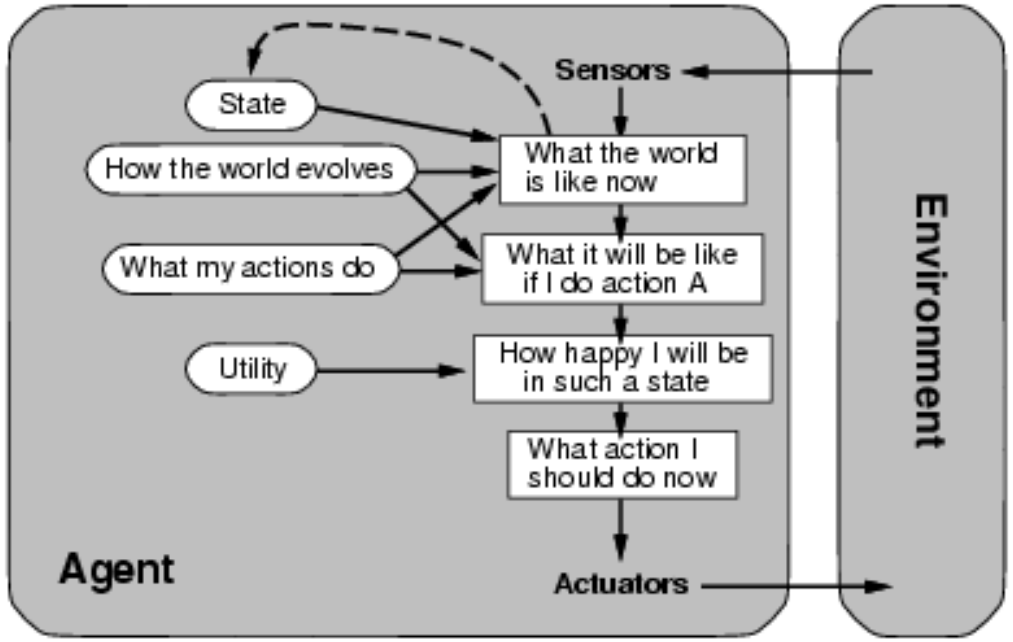
- The knowledge of the current state environment is not always sufficient to decide for an agent to what to do.
- The agent needs to know its goal which describes desirable situations.
- Goal-based agents expand the capabilities of the model-based agent by having the "goal" information.
- They choose an action, so that they can achieve the goal.
- These agents may have to consider a long sequence of possible actions before deciding whether the goal is achieved or not. Such considerations of different scenario are called searching and planning, which makes an agent proactive.

```
function Goal-Based-Agent(percept, goal) returns action
    static : rules, a set of condition-action rules
             state, a description of the current state
    state := Update-State (state, percept)
    rule := Plan-Best-Move(state, rules, goal)
    action := Rule-Action[rule]
    state := Update-State (state, action)
return action
```

Utility-based agents

- We say that if one world state is preferred to another, then it has higher *utility* for the agent.
- Utility is a function that maps a state onto a real number.
  - state → R
- Any rational agent possesses a utility function.

- A utility function maps each state after each action to a real number representing how efficiently each action achieves the goal.

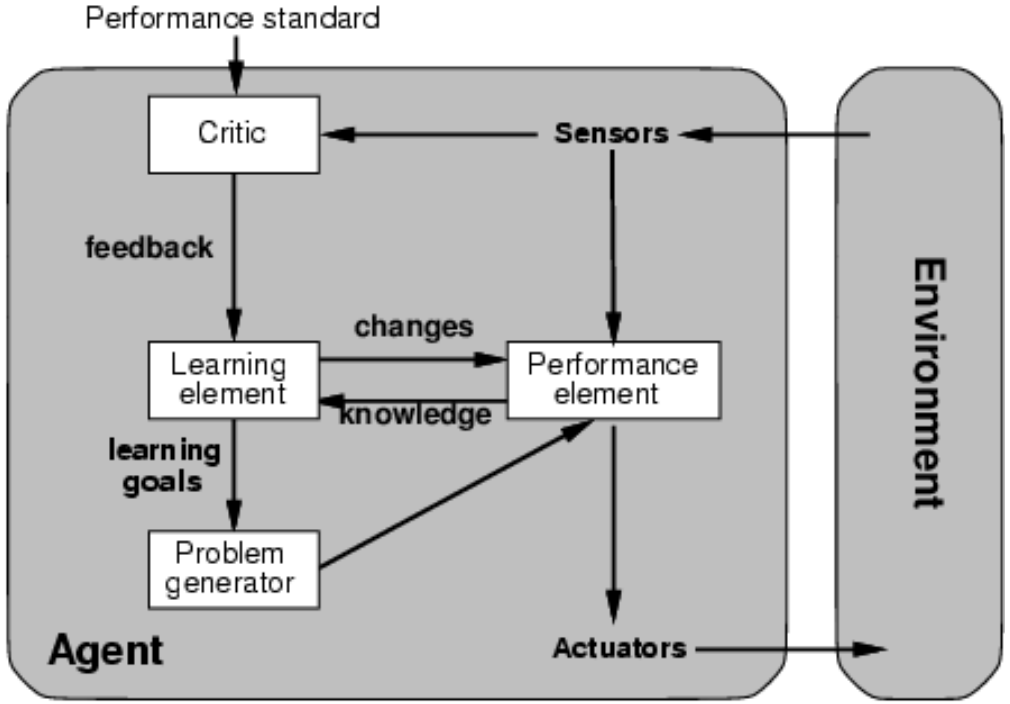


Utility Based Agents

- These agents are similar to the goal-based agent but provide an extra component of utility measurement which makes them different by providing a measure of success at a given state.
- Utility-based agent act based not only goals but also the best way to achieve the goal.
- The Utility-based agent is useful when there are multiple possible alternatives, and an agent has to choose in order to perform the best action.
- The utility function maps each state to a real number to check how efficiently each action achieves the goals.

Learning Agents

- It allows agents to operate in unknown environments and to become more competent.
- learning element
  - responsible for making improvements
- performance element
  - responsible for selecting actions.
- Uses feedback from the **critic** on how the agent is doing.
- **problem generator**
  - responsible for suggesting actions that will lead to new experiences suggest (exploratory actions).



Learning Agents

- A learning agent in AI is the type of agent which can learn from its past experiences, or it has learning capabilities.
- It starts to act with basic knowledge and then able to act and adapt automatically through learning.
- A learning agent has mainly four conceptual components, which are:
  - **Learning element:** It is responsible for making improvements by learning from environment
  - **Critic:** Learning element takes feedback from critic which describes that how well the agent is doing with respect to a fixed performance standard.
  - **Performance element:** It is responsible for selecting external action
  - **Problem generator:** This component is responsible for suggesting actions that will lead to new and informative experiences.
- Hence, learning agents are able to learn, analyze performance, and look for new ways to improve the performance.

Intelligent Agents

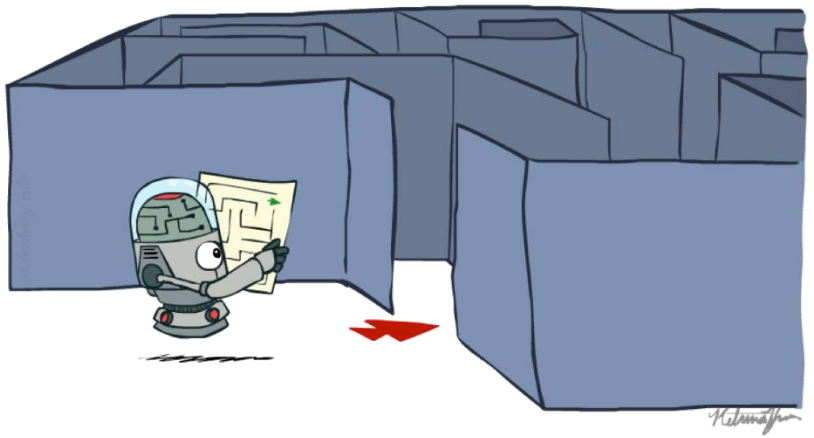
- **Simple reflex agents** respond directly to percepts
- **Model-based reflex agents** maintain internal state to track aspects of the world that are not evident in the current percept.
- **Goal-based agents** act to achieve their goals,
- **Utility-based agents** try to maximize their own expected “happiness.”

Highlights

- Intelligent Agent
- Performance measure
- Different types of Environments
- Agent types
  - Simple reflex agents
  - Reflex agents with state/model
  - Goal-based agents
  - Utility-based agents

Problem Solving Using Search

Introduction to State Space Search



Introduction

- Suppose an agent can execute several actions immediately in a given state
- It doesn't know the utility of these actions
- Then, for each action, it can execute a sequence of actions until it reaches the goal
- The immediate action which has the best sequence (according to the performance measure) is then the solution
- Finding this sequence of actions is called search, and the agent which does this is called the problem-solver.
- NB: Its possible that some sequence might fail, e.g., getting stuck in an infinite loop, or unable to find the goal at all.


T 69
LISDOONVARNA 10
BOTHAIR NA TRA
LISDOONVARNA 19
VIA COAST ROAD
51 KILLIMER CAR FERRY
KORA FINNE
COROFIN 15
An Pcan Sorca
TEA & GARDEN ROOMS
GREGANS
5 CASTLE HOTEL
RUSHEEN LODGE
B & B
AILLWEE CAVE
BALLYVAUGHAN
OPEN TO VISITORS
Aran Islands
Doolin Ferries
Burren
Dolmen
West Clare
Cycleways
2 NEWTOWN CASTLE & TRAIL



Problem-Solving Agent

- **Problem solving agent** is a **goal-based agent** that decides what to do by **systematically** finding sequences of actions that lead to desirable states (goal)

- Many problems can be represented as a **set of states** and a **set of rules** of how one state is transformed to another.

Examples

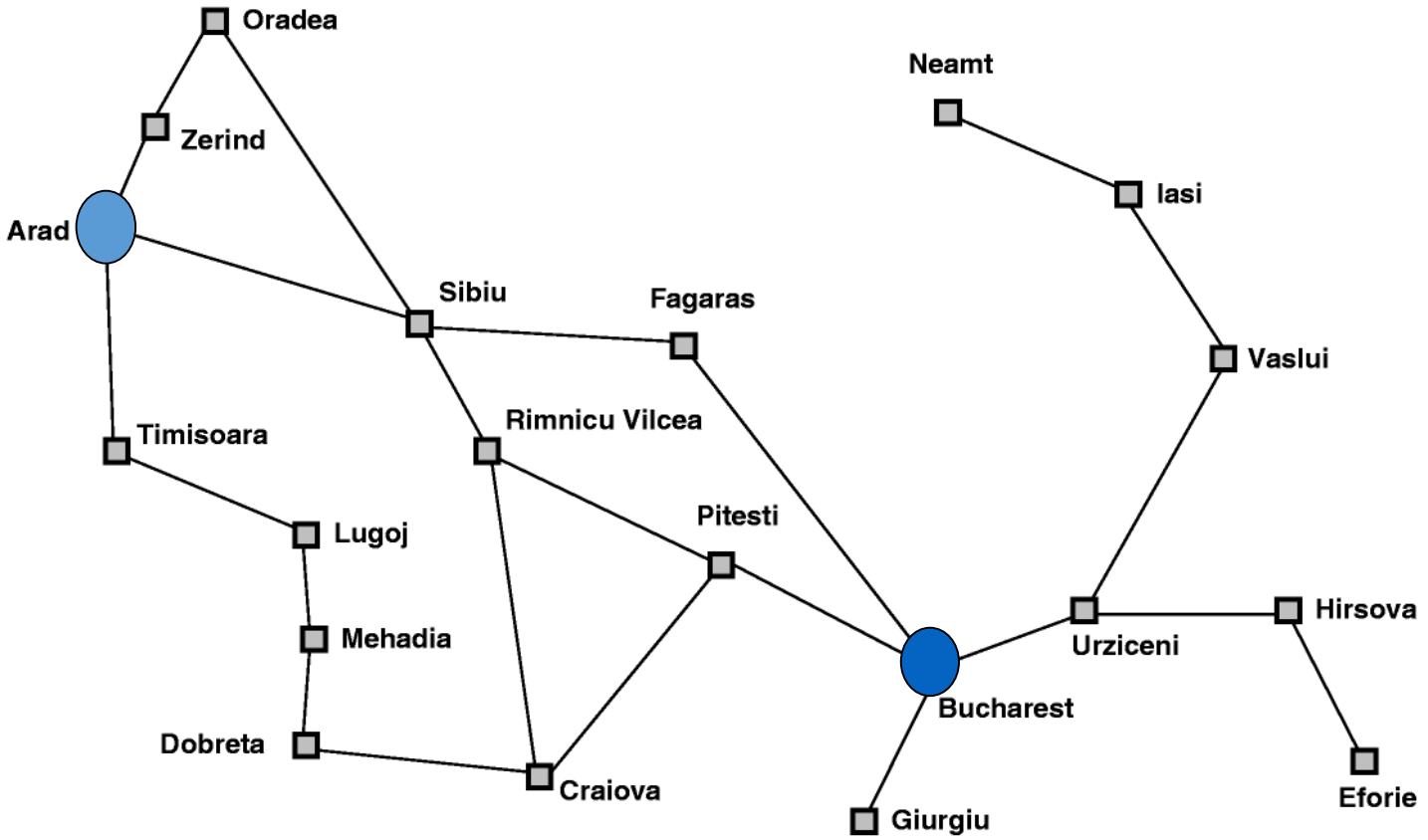
- Getting from home to University
  - **START**: home location
  - **GOAL**: University CS Department.
  - **OPERATORS**: move one block, turn
- Loading a moving truck
  - **START**: apartment full of boxes and furniture
  - **GOAL**: empty apartment, all boxes and furniture in the truck
  - **OPERATORS**: select item, carry item from apartment to truck, load item
- Getting settled
  - **START**: items randomly distributed over the place
  - **GOAL**: satisfactory arrangement of items
  - **OPERATORS**: select item, move item

Example Problems

- **Toy Problems**
  - vacuum world
  - 8-puzzle
  - 8-queens
  - crypt arithmetic
  - vacuum agent
  - missionaries and cannibals
- **Real-world Problems**
  - route finding
  - touring problems
    - traveling salesperson
  - VLSI layout
  - robot navigation
  - assembly sequencing
  - Web search

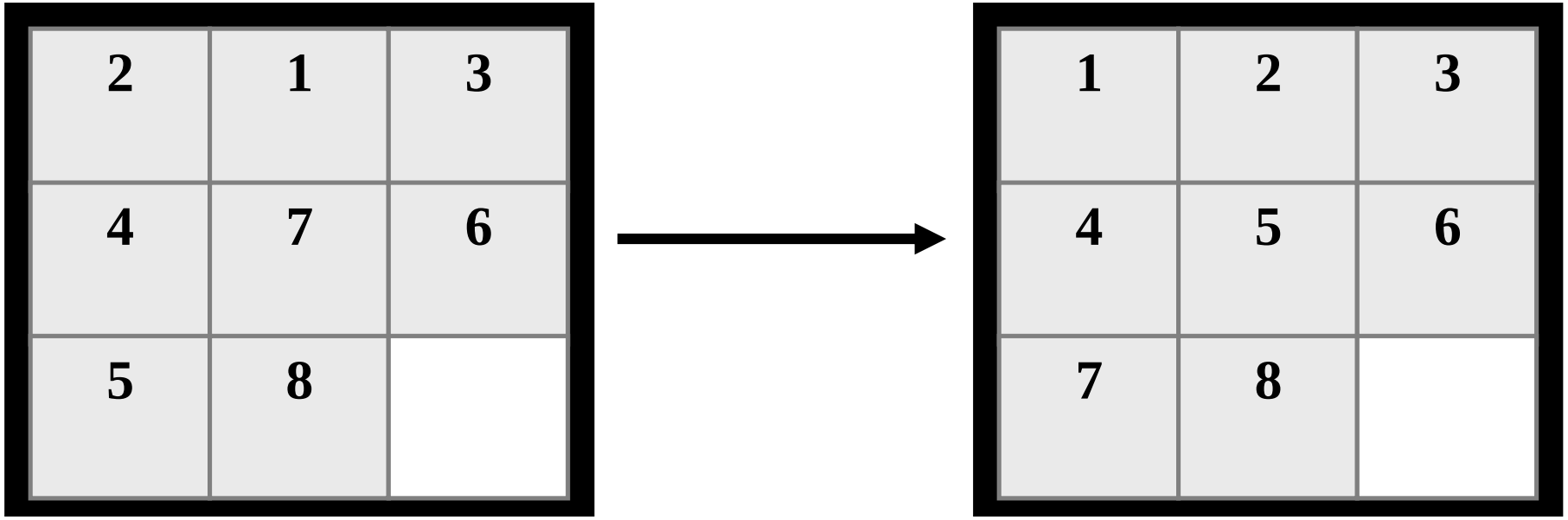
Classical AI Search Problem

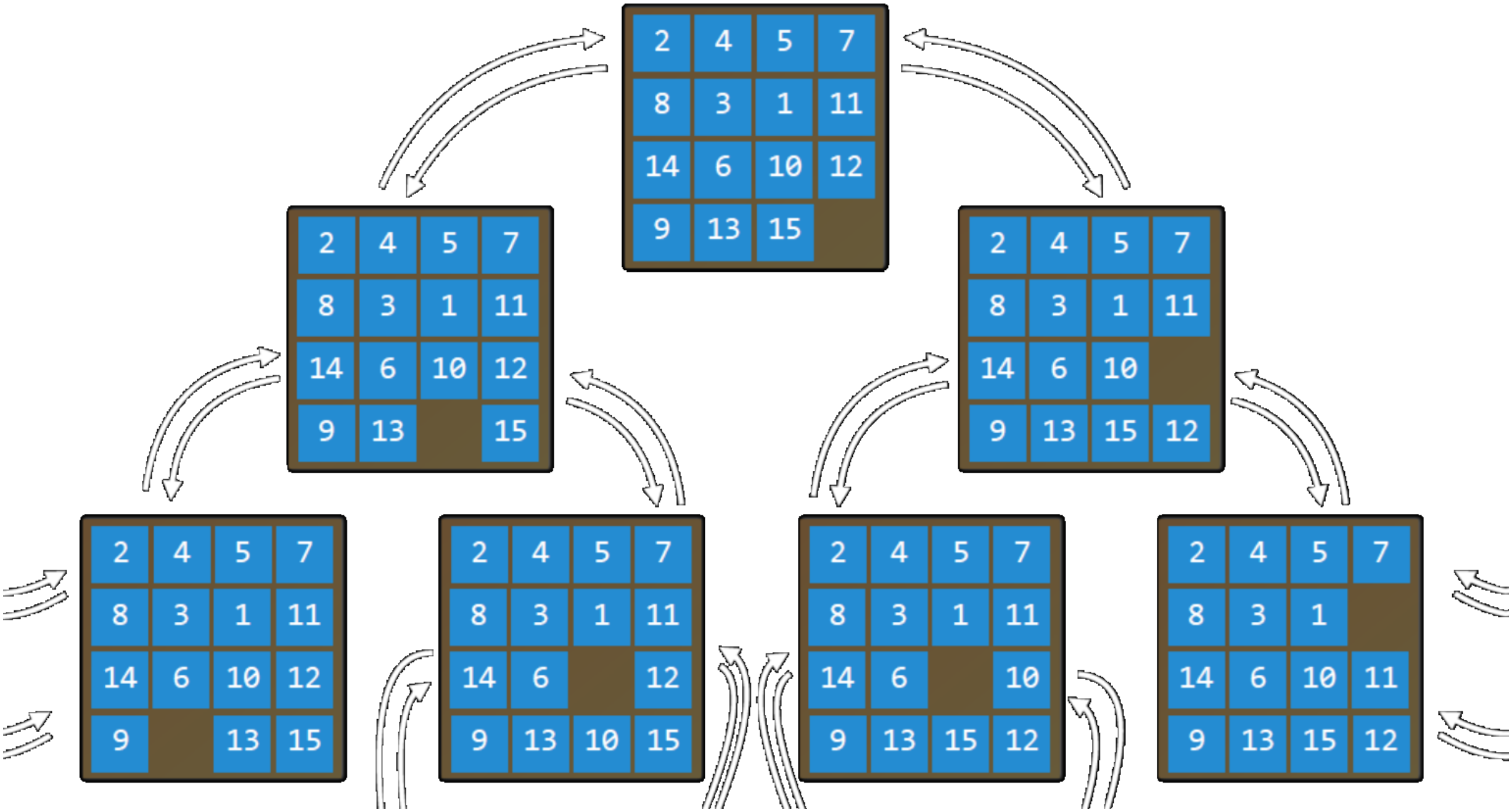
- Touring Agent Problem (Map Search / Navigation)



Classical AI Search Problem

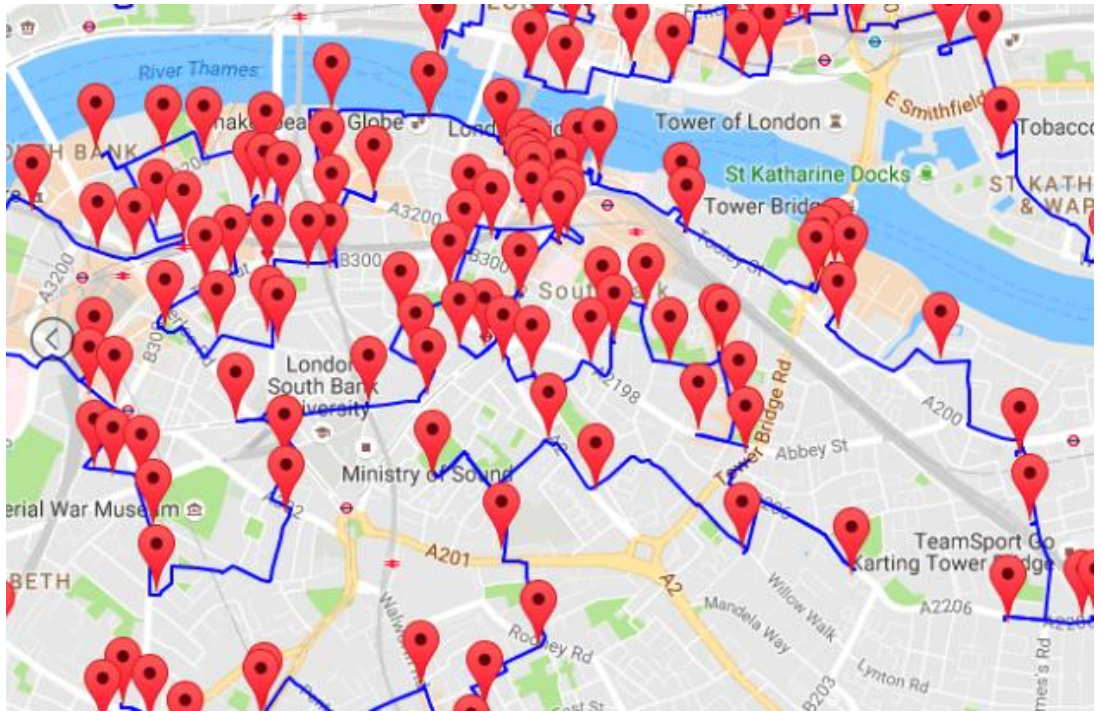
- 8-Puzzle




2 4 5 7
8 3 1 11
14 6 10 12
9 13 15
2 4 5 7
8 3 1 11
14 6 10 12
9 13 15
2 4 5 7
8 3 1 11
14 6 10
9 13 15 12
2 4 5 7
8 3 1 11
14 6 10 12
9 13 15
2 4 5 7
8 3 1 11
14 6 12
9 13 10 15
2 4 5 7
8 3 1 11
14 6 10
9 13 15 12
2 4 5 7
8 3 1
14 6 10 11
9 13 15 12

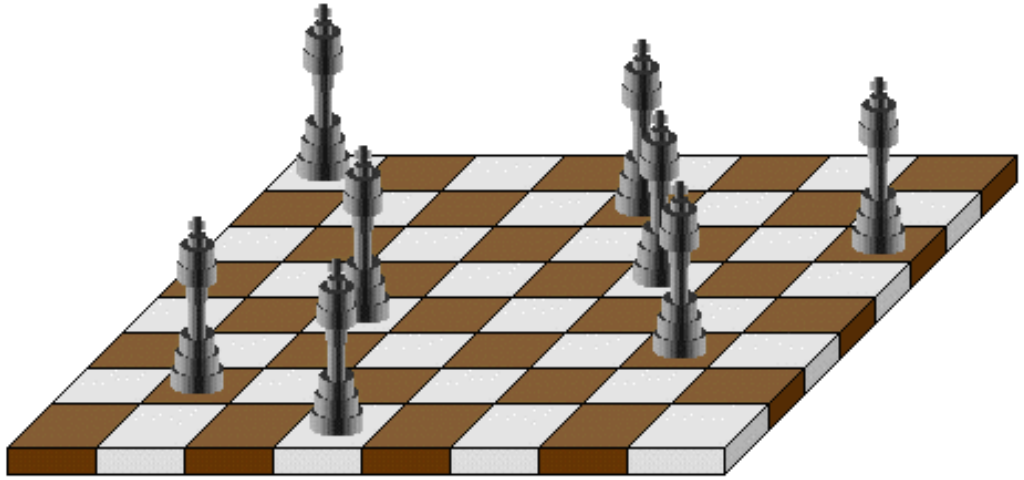
Classical AI Search Problem

- **Travelling Salesperson Problem (TSP)**
  - Suppose a salesman has five cities to visit and ten must return home.
  - The goal of the problem is to find the shortest path for salesman to travel, visiting each city, and then returning to the starting city.

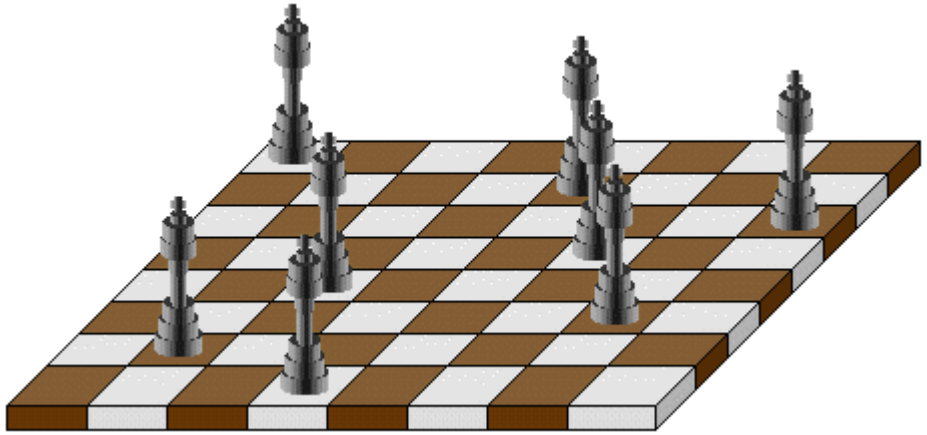


Classical AI Search Problem

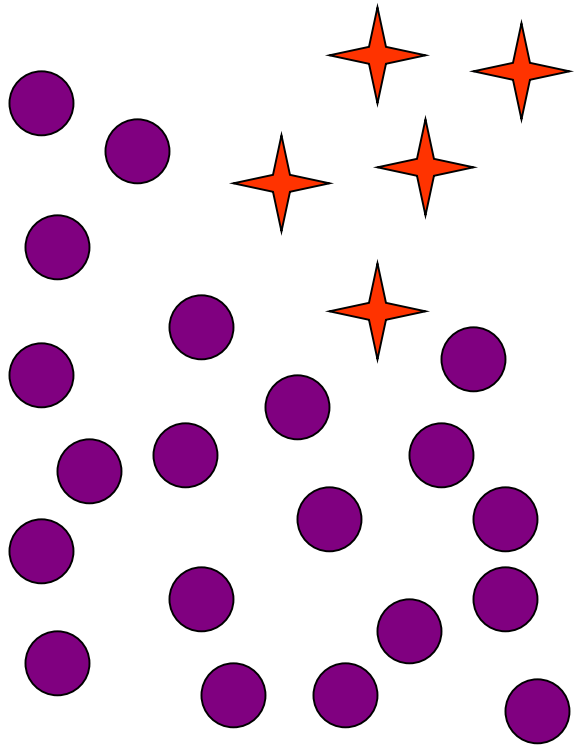
- N-Queens
  - Problem of placing *n* chess queens on an *n*×*n* chessboard so that no two queens attack each other
  - A solution requires that no two queens share the same row, column, or diagonal

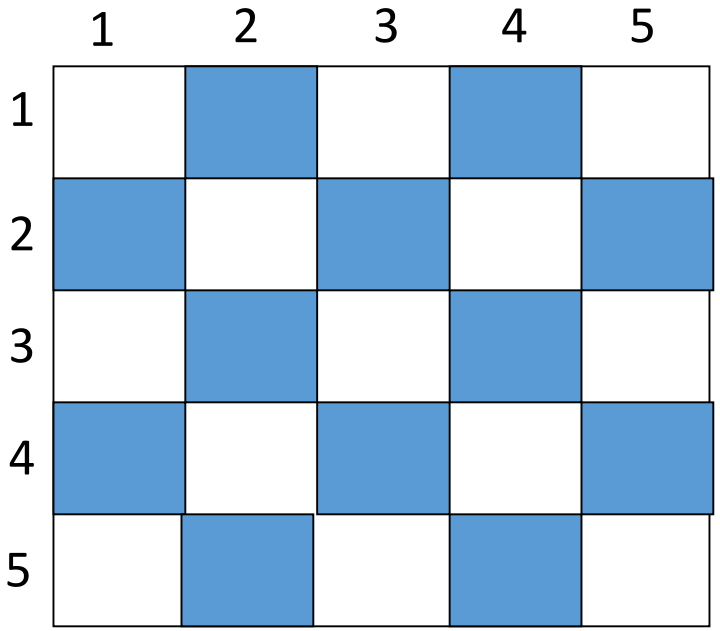


Classic AI Search Problems

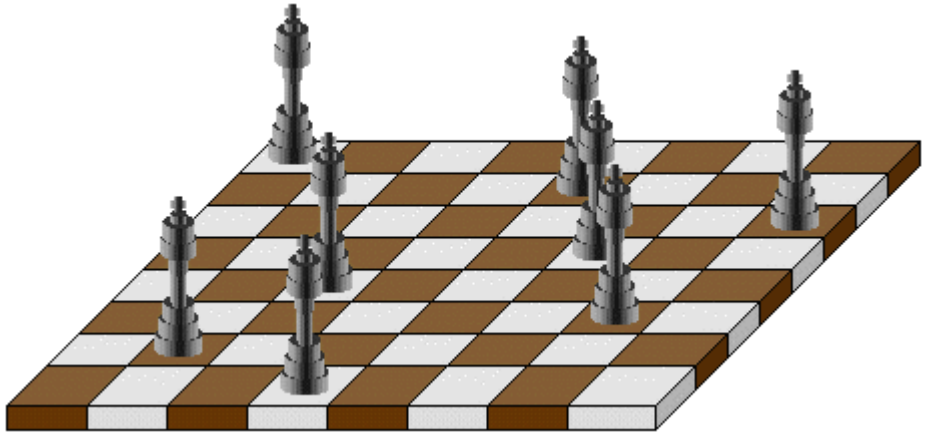


- 5-Queens:

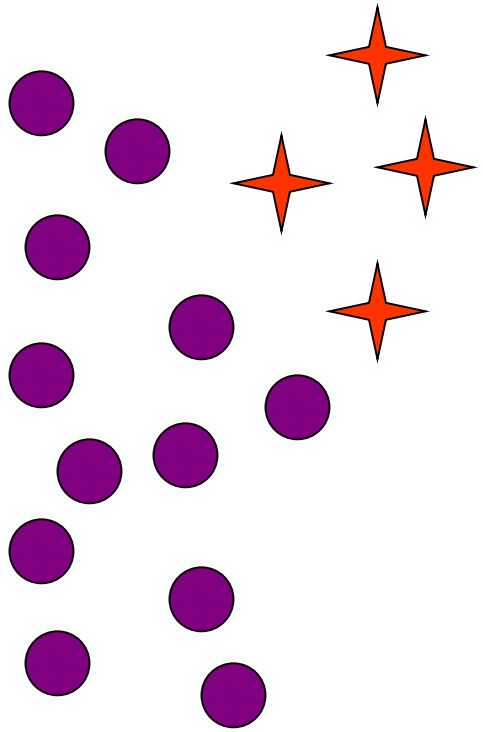


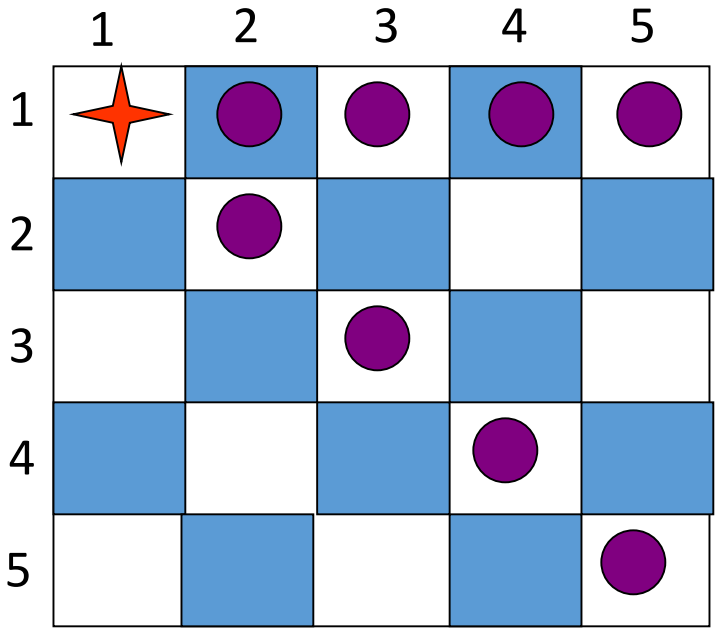


Classic AI Search Problems

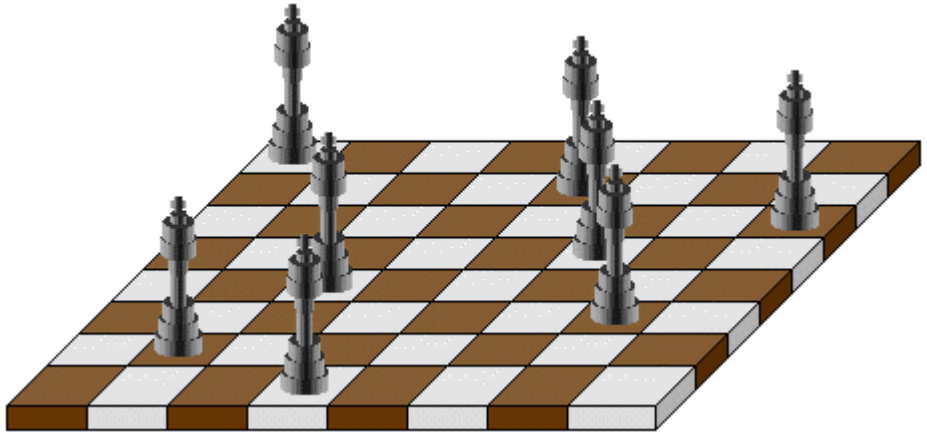


- 5-Queens:

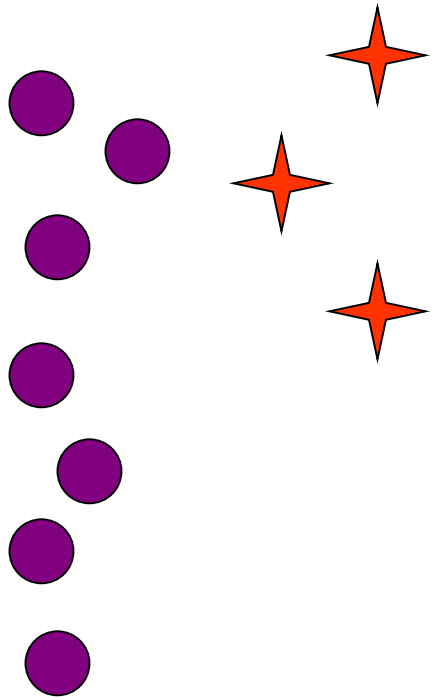


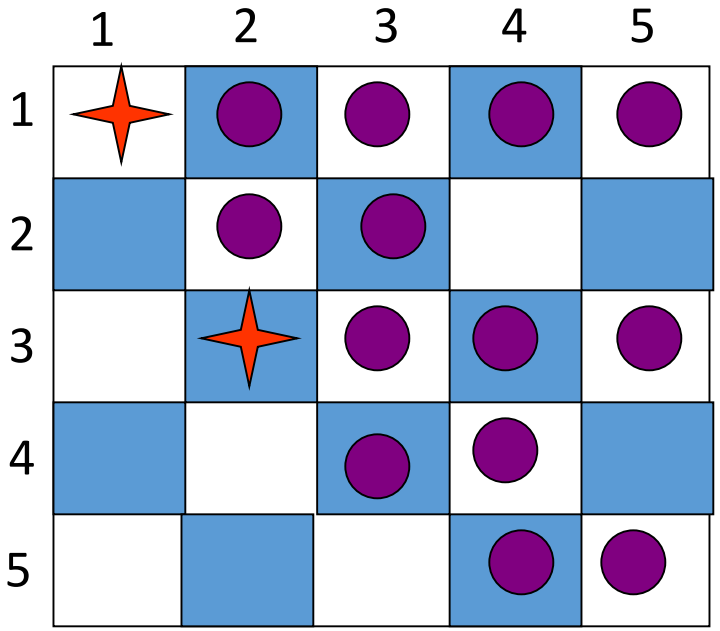


Classic AI Search Problems

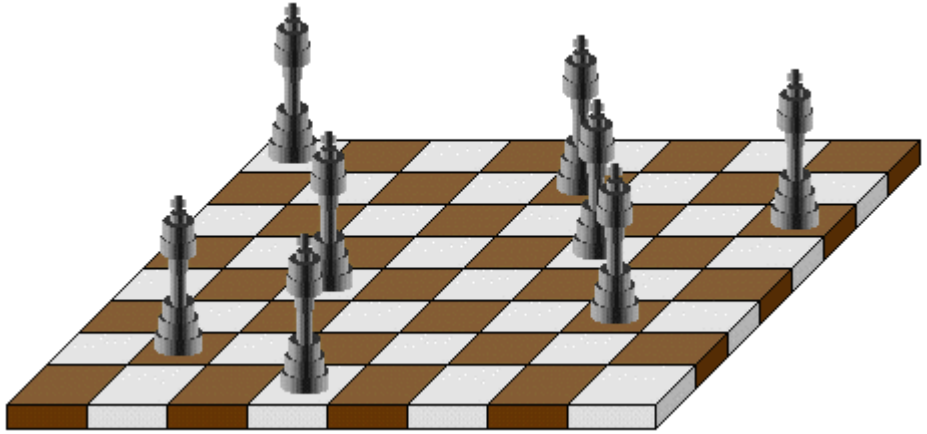


- 5-Queens:

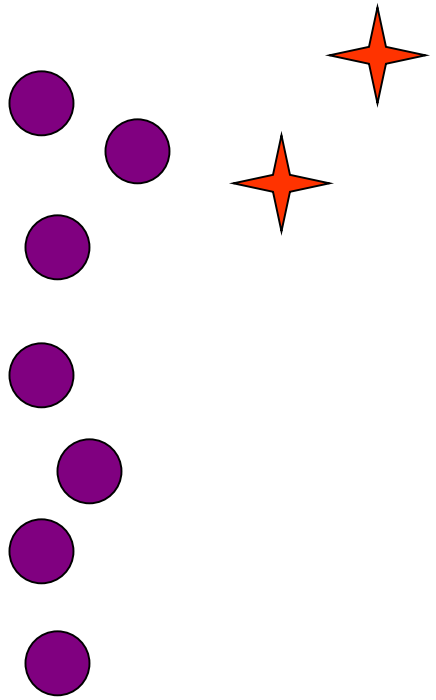


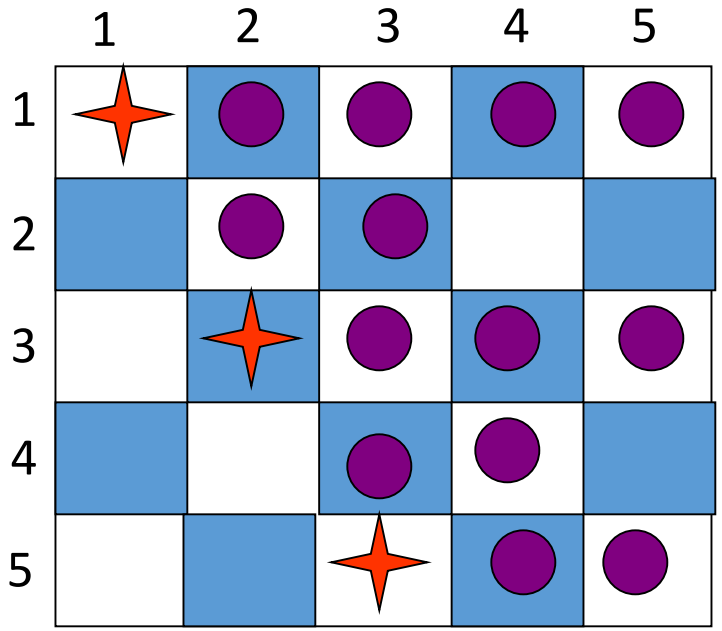


Classic AI Search Problems

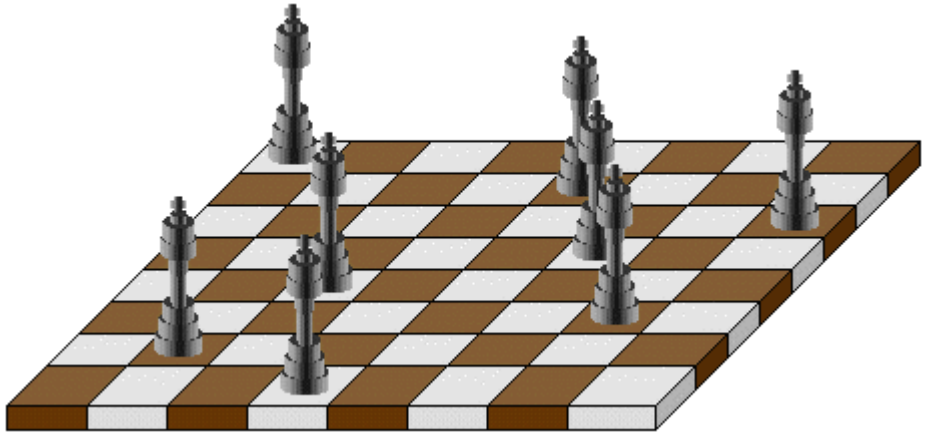


- 5-Queens:

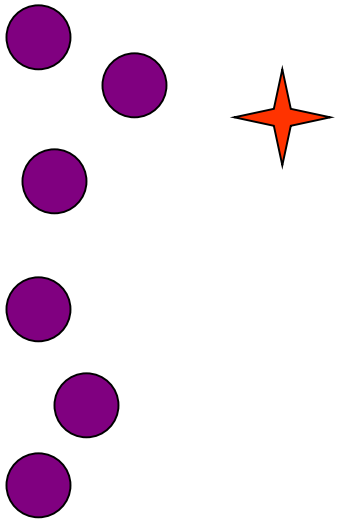


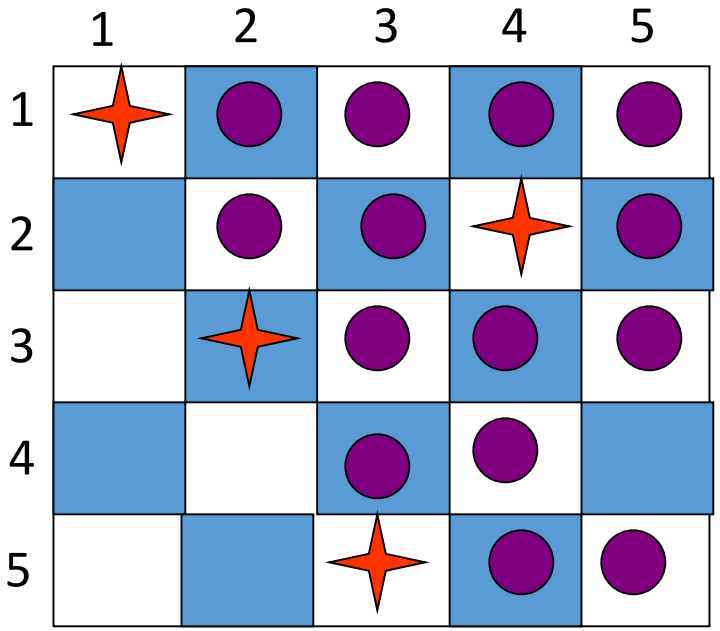


Classic AI Search Problems

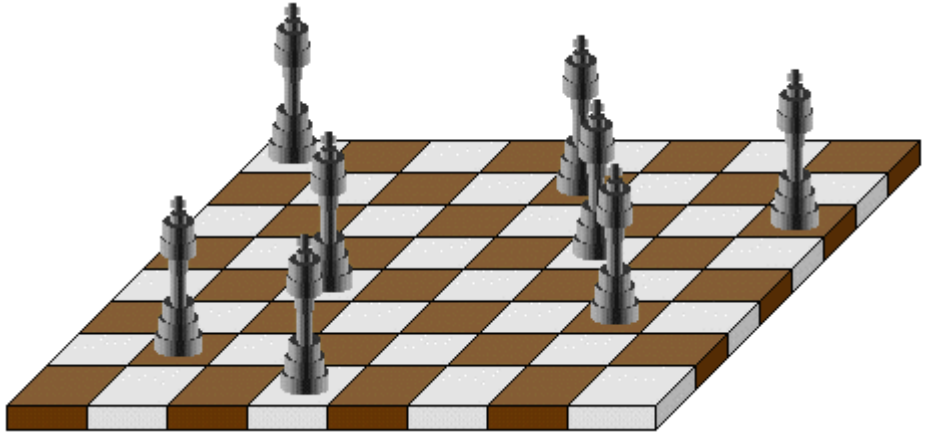


- 5-Queens:

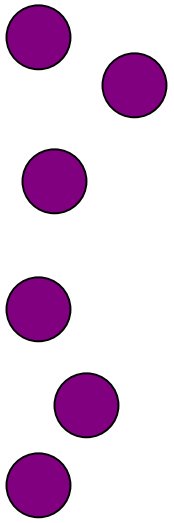




Classic AI Search Problems

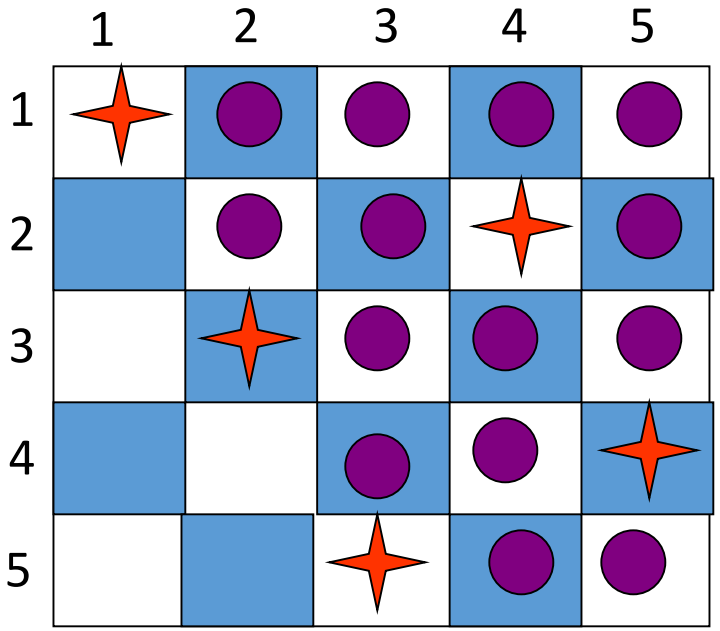


- 5-Queens:



Solution !!

No Queen is under Attack



River Crossing Problem

- A farmer wishes to carry a wolf, a duck and corn across a river, from the south to the north shore.
  - The farmer has a small rowing boat.
  - The boat can only carry at most the farmer and one other item.
  - If left unattended the wolf will eat the duck and the duck will eat the corn.

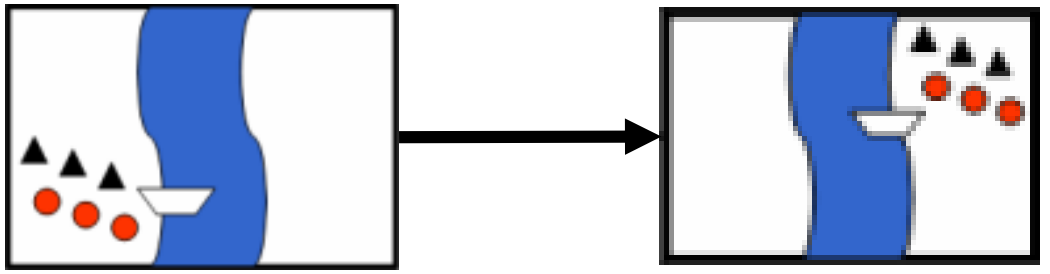
- How can the farmer safely transport the wolf, the duck and the corn to the opposite shore?



Missionaries and cannibals

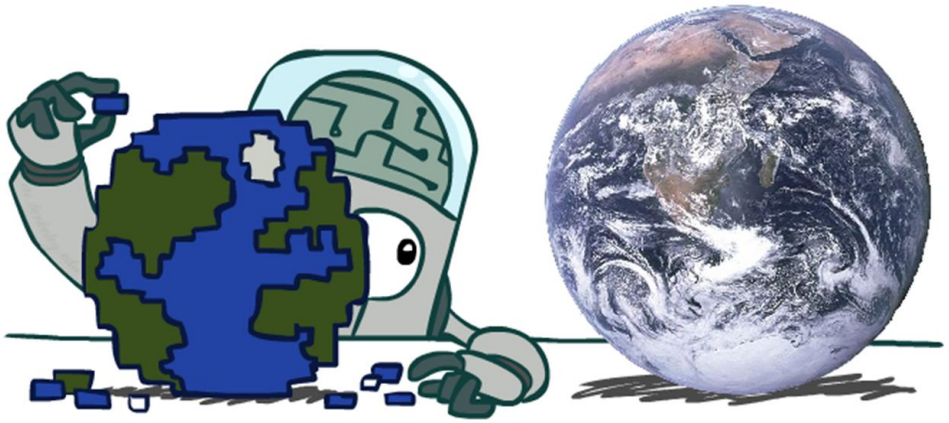
- Three missionaries and three cannibals are on the left bank of a river.
- There is one boat which can hold one or two people.
- Find a way to get everyone to the right bank, without ever leaving a group of missionaries in one place outnumbered by cannibals in that place.





Initial State

Goal State



Problem Solving by Searching

Problem Formulation

Search in AI

- Search in Data Structures
  - You're given an existent tree.
  - You search it in different orders.
  - It resides in memory.
- Search in Artificial Intelligence
  - The tree does not exist.
  - You have to generate it as you go.
  - For realistic problems, it does not fit in memory.

Problem Formulation

A **Problem Space** consists of

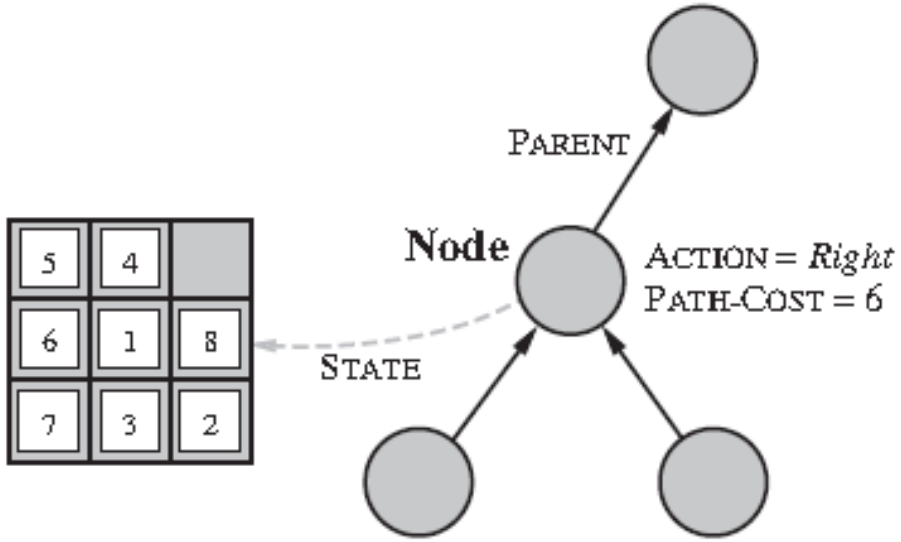
- The current state of the world (**initial state**)
- A description of the actions we can take to transform one state of the world into another (**operators or successor function**).
- A description of the desired state of the world (**goal state**), this could be implicit or explicit.
- A **solution** consists of the goal state, or a path to the goal state.

Problem Formulation

- Initial state
- Actions (operators, successor functions)
- State space
- Path
- Goal test
- Solution
- Path cost
- Total cost

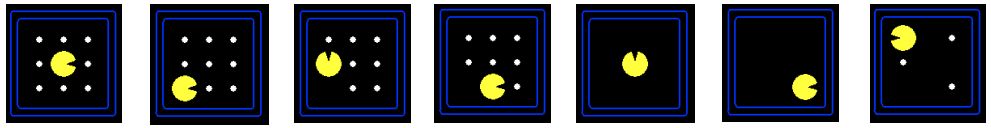
Representation of a Search/State Space

- A convenient way of representing search spaces is as a graph
  - **States** are nodes
  - **Actions** are edges
  - **Initial state** is root
  - **Solution** is path from root to goal node
  - Edges sometimes have associated costs
  - States resulting from operator are children

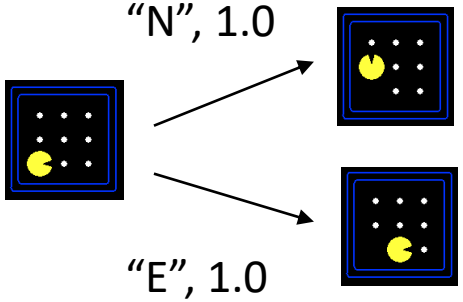


State Space Representation

- A search problem consists of:
  - A state space



  - A successor function (with actions, costs)



  - A start state and a goal test

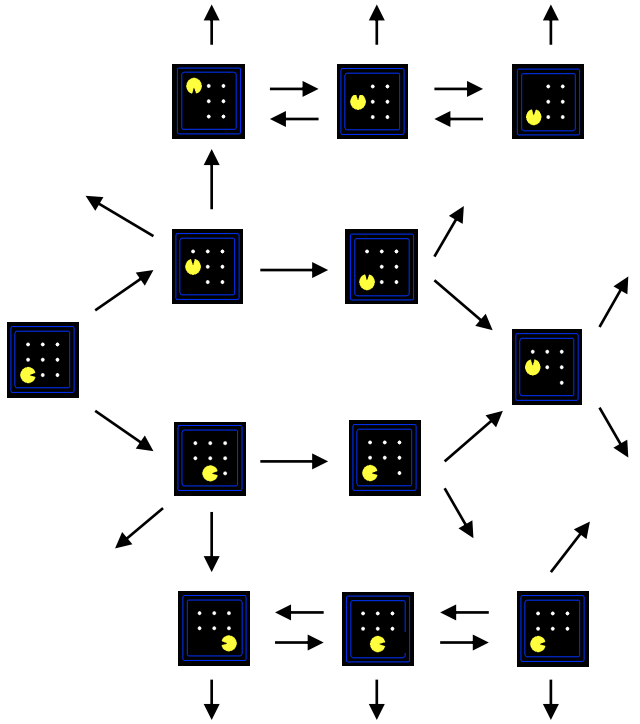
- A solution is a sequence of actions (a plan) which transforms the start state to a goal state

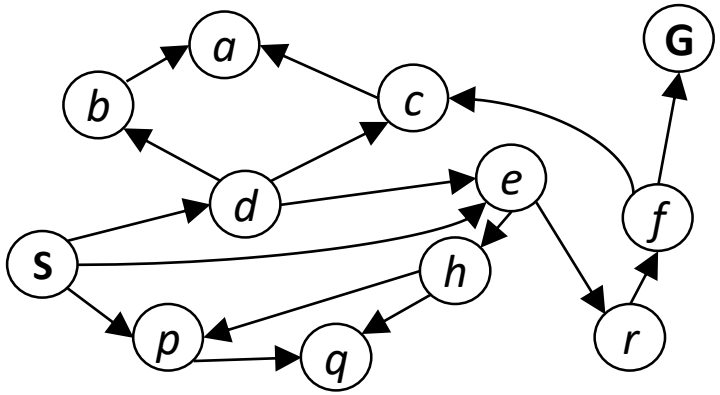
State Space Graphs

- State space graph: A mathematical representation of a search problem
  - Nodes are (abstracted) world configurations
  - Arcs represent successors (action results)
  - The goal test is a set of goal nodes (maybe only one)

- In a state space graph, each state occurs only once!

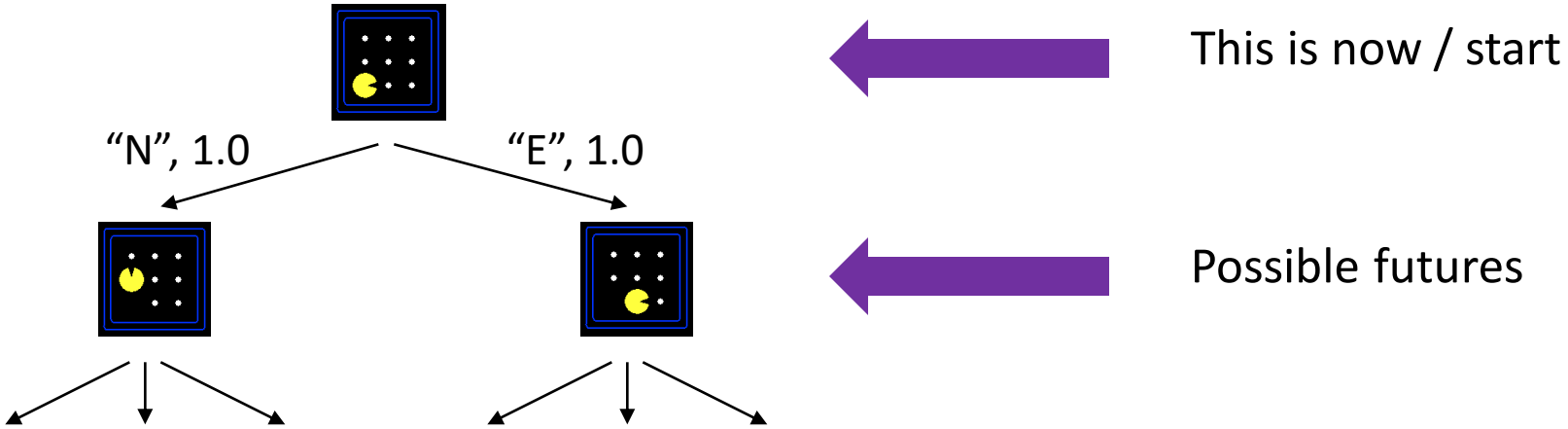
- We can rarely build this full graph in memory (it's too big), but it's a useful idea





Tiny search graph for a tiny search problem

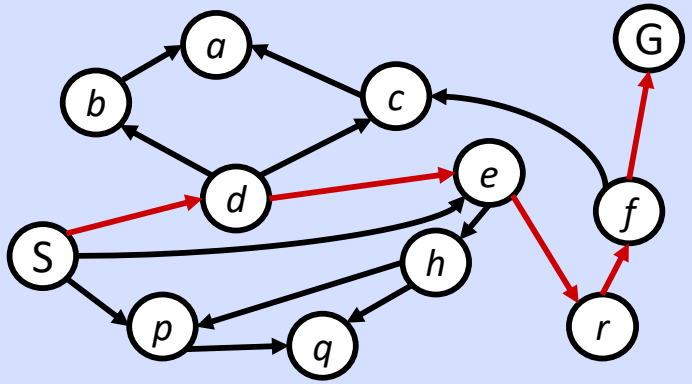
Search Trees



- A search tree:
  - A "what if" tree of plans and their outcomes
  - The start state is the root node
  - Children correspond to successors
  - Nodes show states, but correspond to PLANS that achieve those states
  - For most problems, we can never actually build the whole tree

State Space Graphs vs. Search Trees

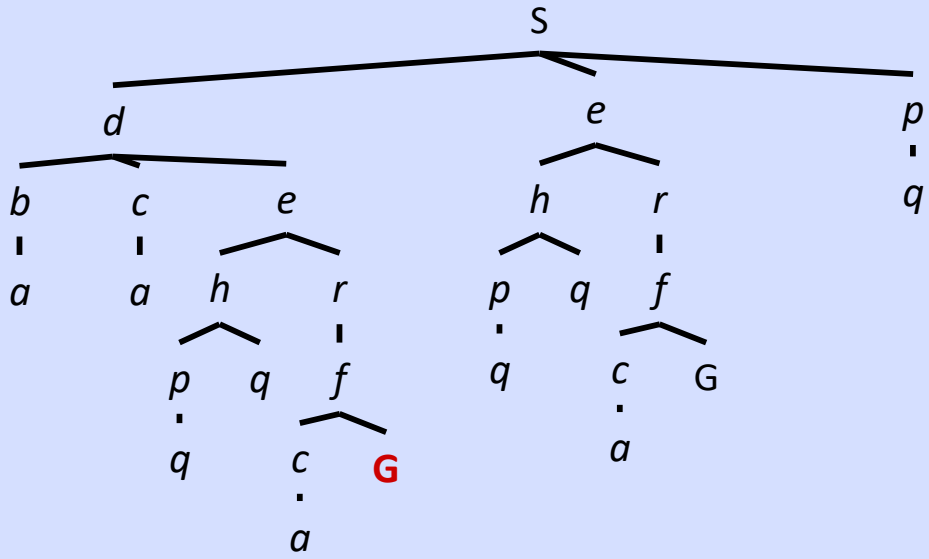
State Space Graph



Each NODE in the search tree is an entire PATH in the state space graph.

We construct both on demand – and we construct as little as possible.

Search Tree

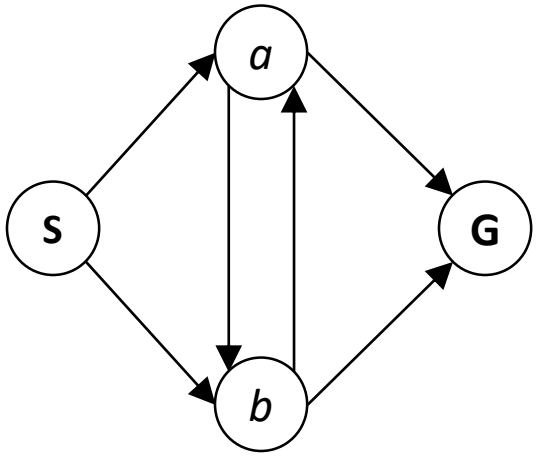


Approach

- Start with a **frontier** that contains the initial state.
- Repeat:
  - If the frontier is empty, then no solution.
  - Remove a node from the frontier.
  - If node contains goal state, return the solution.
  - **Expand** node, add resulting nodes to the frontier.

Quiz: State Space Graphs vs. Search Trees

Consider this 4-state graph:



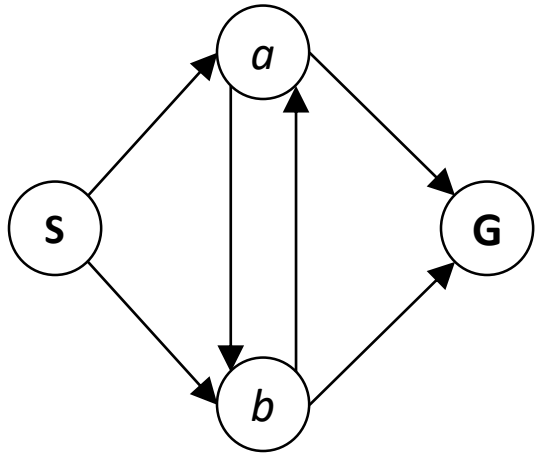
How big is its search tree (from S)?



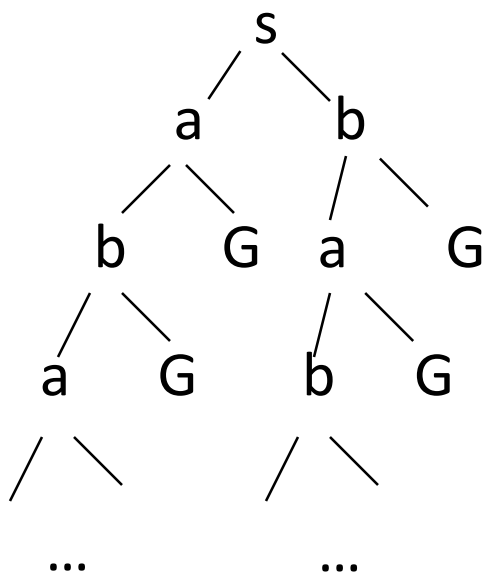
Important: Lots of repeated structure in the search tree!

Quiz: State Space Graphs vs. Search Trees

Consider this 4-state graph:



How big is its search tree (from S)?





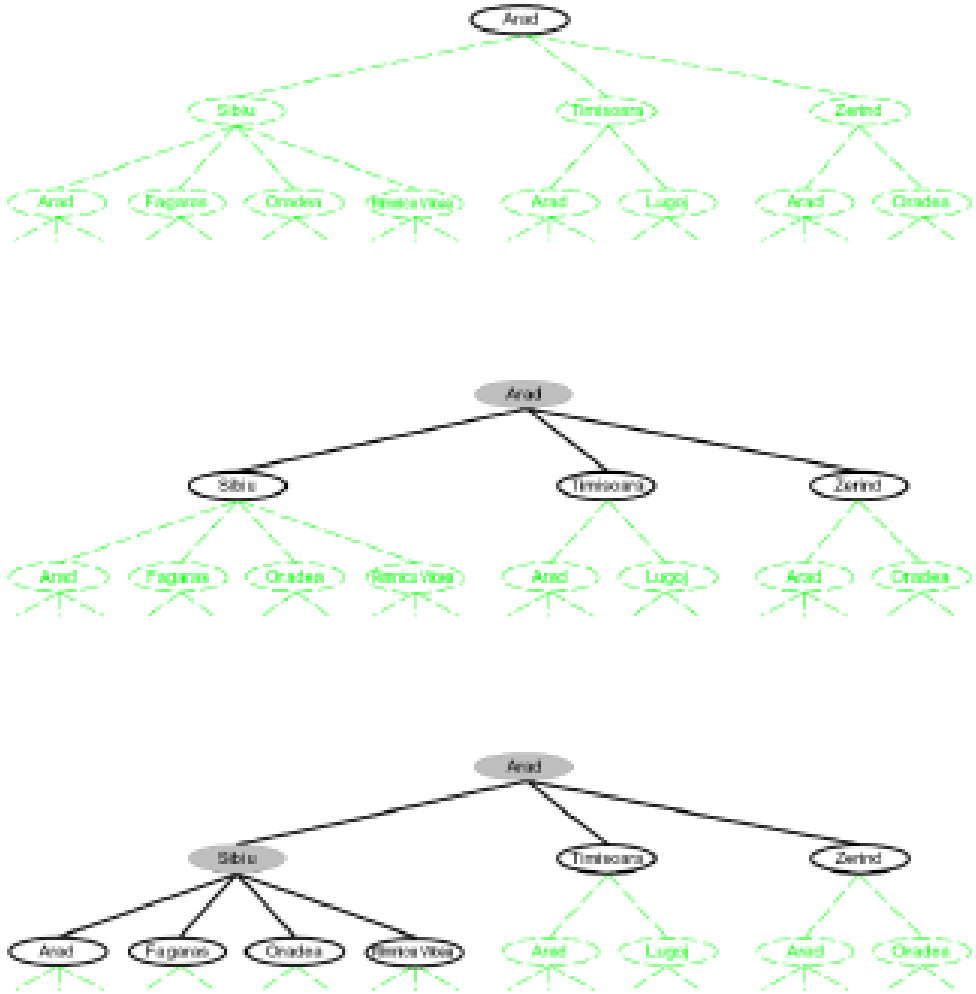
Important: Lots of repeated structure in the search tree!

Revised Approach

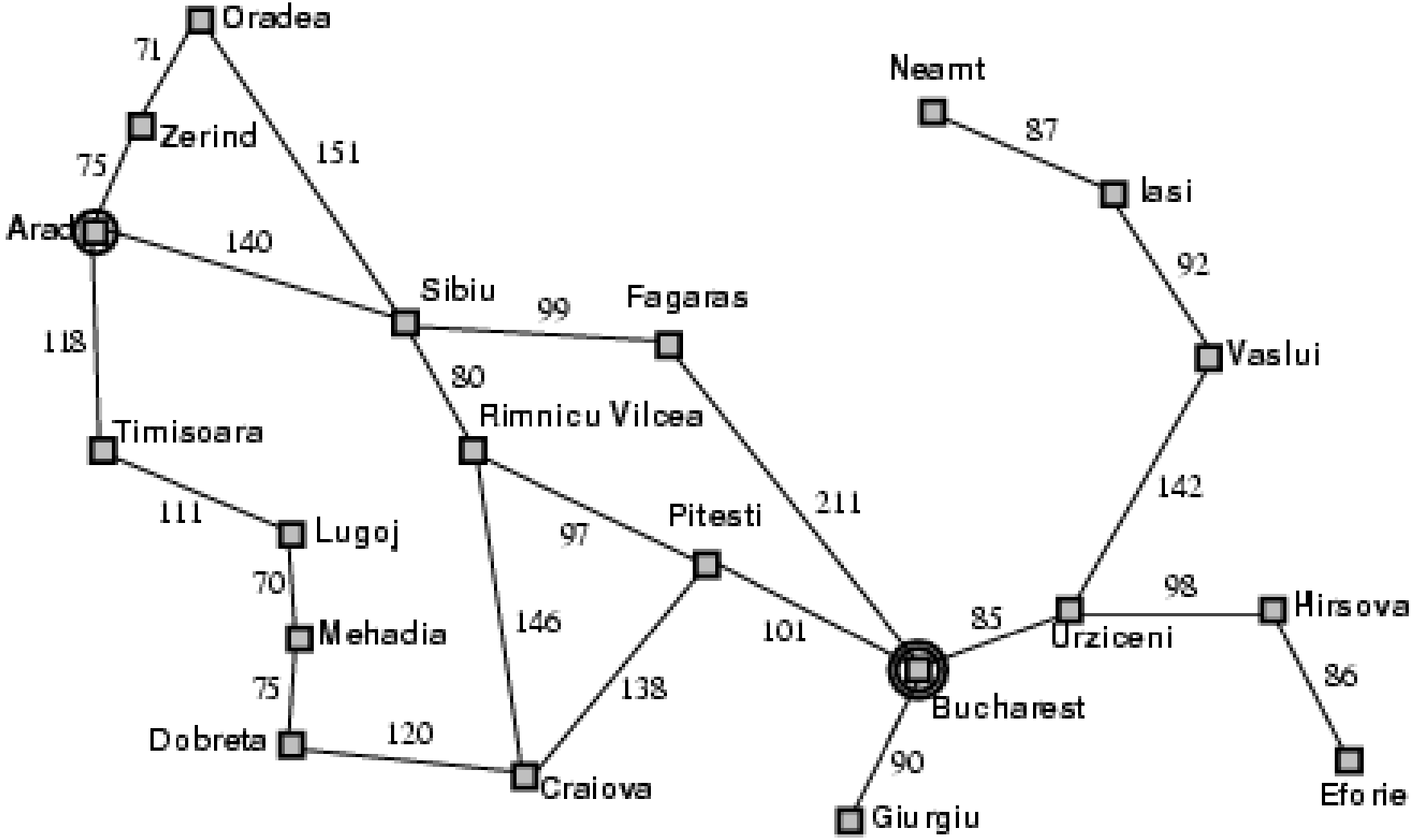
- Start with a **frontier** that contains the initial state.
- Start with an empty **explored set**.
- Repeat:
  - If the frontier is empty, then no solution.
  - Remove a node from the frontier.
  - If node contains goal state, return the solution.
  - Add the node to the explored set.
  - **Expand** node, add resulting nodes to the frontier if they aren't already in the frontier or the explored set.

Searching with a Search Tree

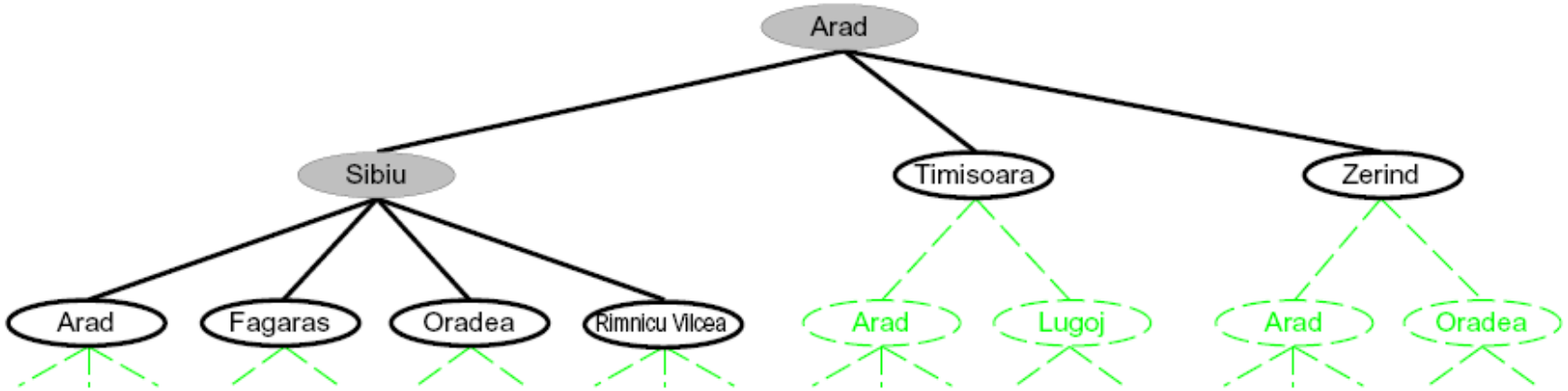
- Search:
- (1) Expand out potential plans (tree nodes)
- (2) Maintain a fringe of partial plans under consideration.
- (3) Try to expand as few tree nodes as possible



Example: Romania



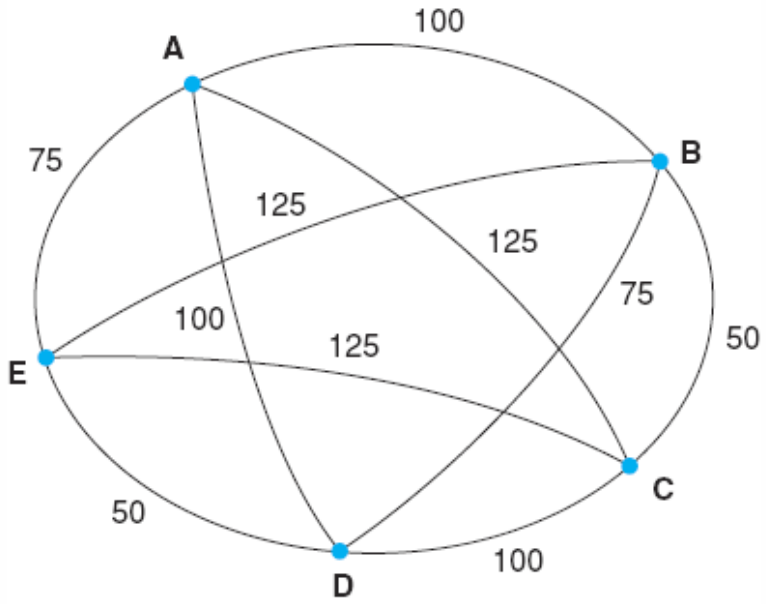
Searching with a Search Tree



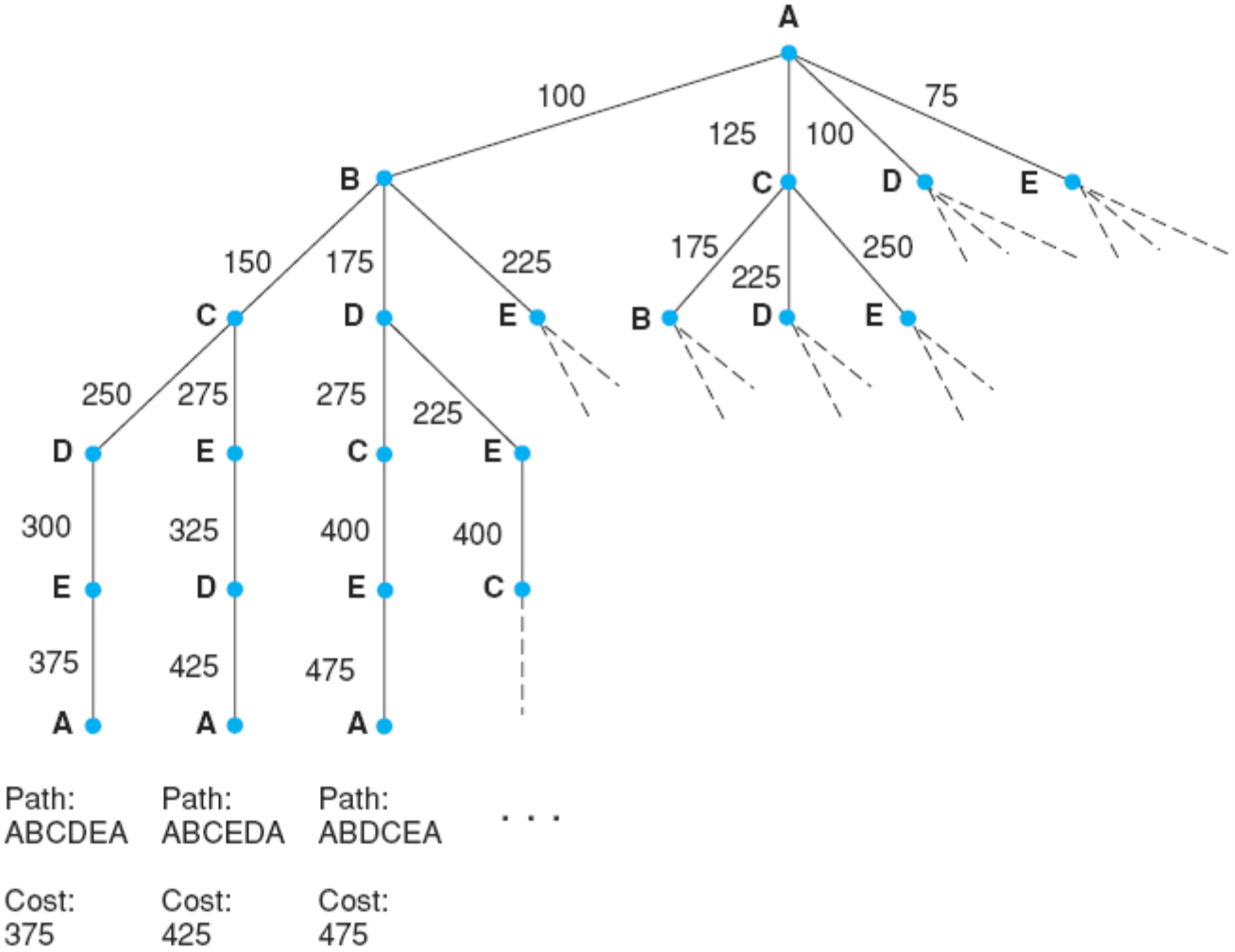
- Search:
  - Expand out potential plans (tree nodes)
  - Maintain a fringe of partial plans under consideration
  - Try to expand as few tree nodes as possible

Problem Formulation: TSP

- **Travelling Salesperson Problem (TSP)**
  - Suppose a salesman has five cities to visit and then must return home.
  - The goal of the problem is to find the shortest path for salesman to travel, visiting each city, and then returning to the starting city.



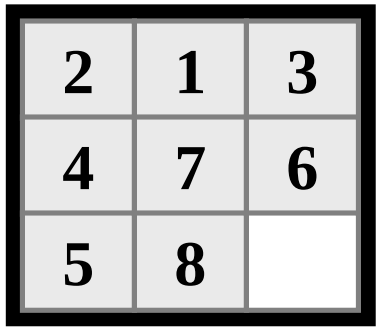
Search for the travelling salesperson problem. Each arc is marked with the total weight of all paths from the start node (A) to its endpoint.



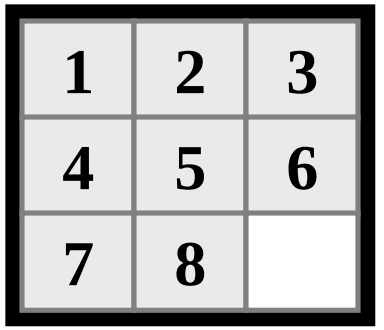
Luger: Artificial Intelligence, 6th edition. © Pearson Education Limited, 2009

Problem Formulation: 8-Puzzle Problem

Initial State	Operators	Goal State



Slide blank square left.
Slide blank square right.
….



Problem Formulation: 8-Puzzle Problem

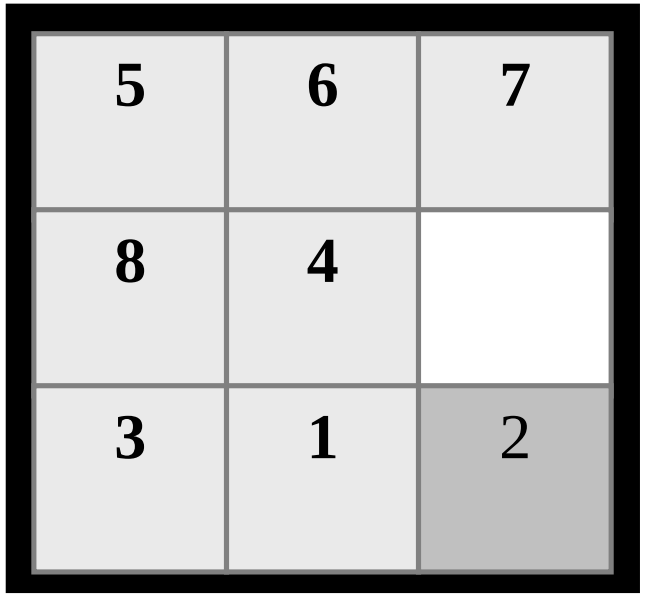
Representing states:

- **For the 8-puzzle**

- 3 by 3 array
  - 5, 6, 7
  - 8, 4, BLANK
  - 3, 1, 2

- A vector of length nine
  - 5, 6, 7, 8, 4, BLANK, 3, 1, 2

- A list of facts
  - Upper_left = 5
  - Upper_middle = 6
  - Upper_right = 7
  - Middle_left = 8

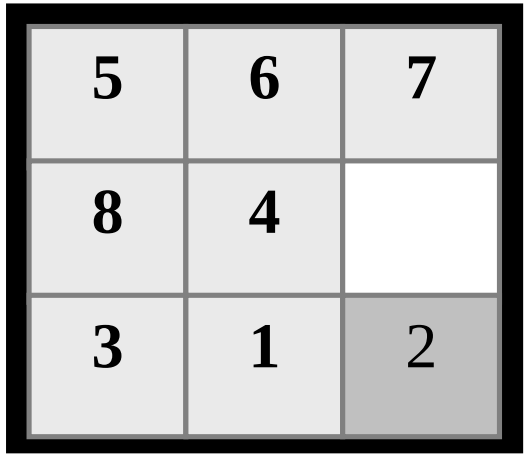


Problem Formulation: 8-Puzzle Problem

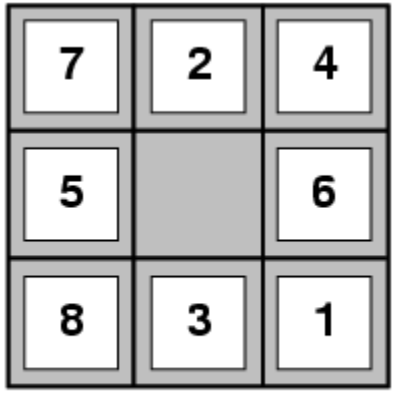
- Specifying operators
  - There are often many ways to specify the operators, some will be much easier to implement...

- **Move 1 left**
- **Move 1 right**
- **Move 1 up**
- **Move 1 down**
- **Move 2 left**
- **Move 2 right**
- **Move 2 up**
- **Move 2 down**
- **Move 3 left**
- **Move 3 right**
- **Move 3 up**
- **Move 3 down**
- **Move 4 left**
- …

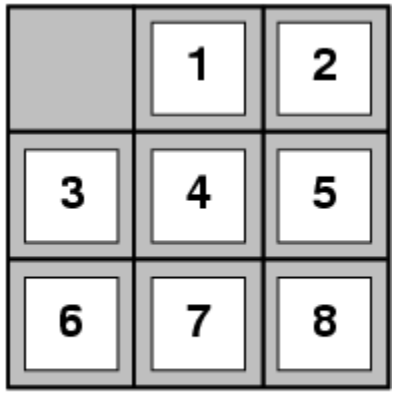
- **Move Blank left**
- **Move Blank right**
- **Move Blank up**
- **Move Blank down**



Problem Formulation: **8-Puzzle Problem**



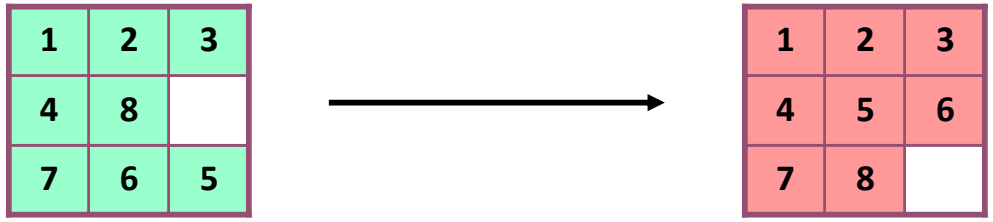
Start State



Goal State

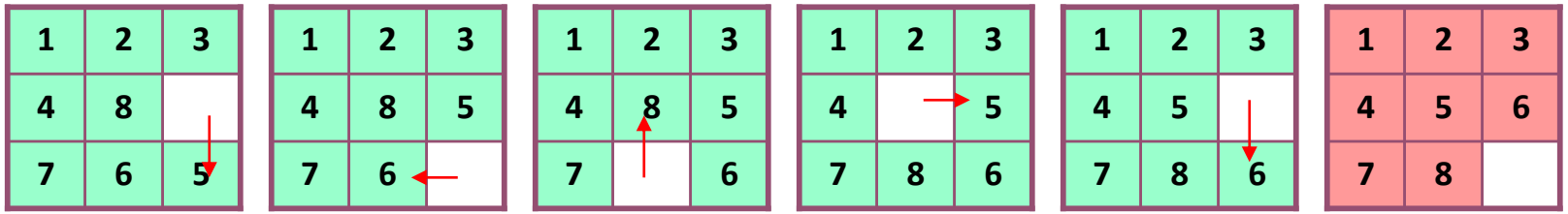
- states locations of tiles
- actions move blank left, right, up, down
- goal test goal state (given)
- path cost 1 per move

Problem Formulation: **8-Puzzle Problem**



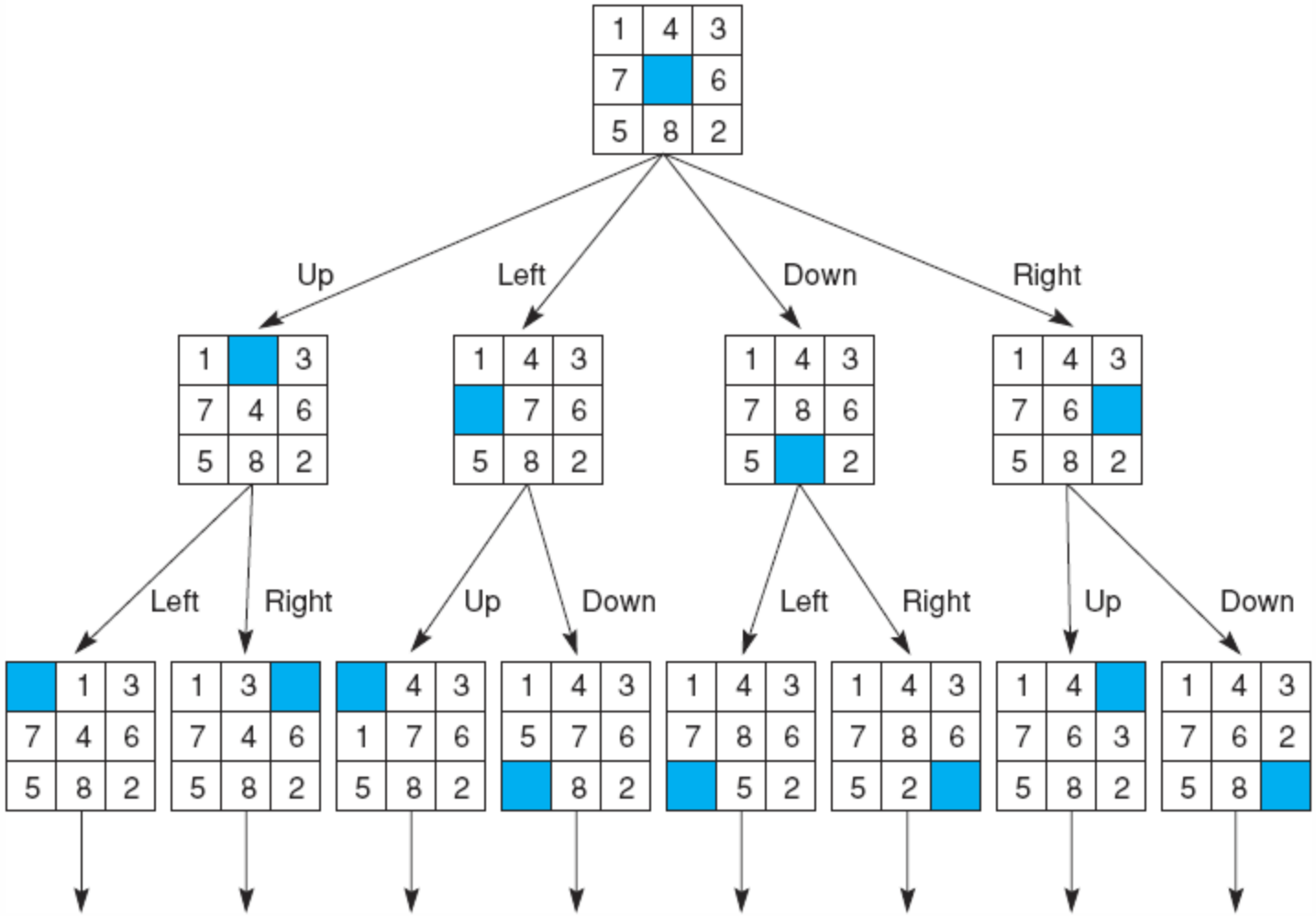
Initial state **Goal state**

- Operators: ***slide blank up, slide blank down, slide blank left, slide blank right***

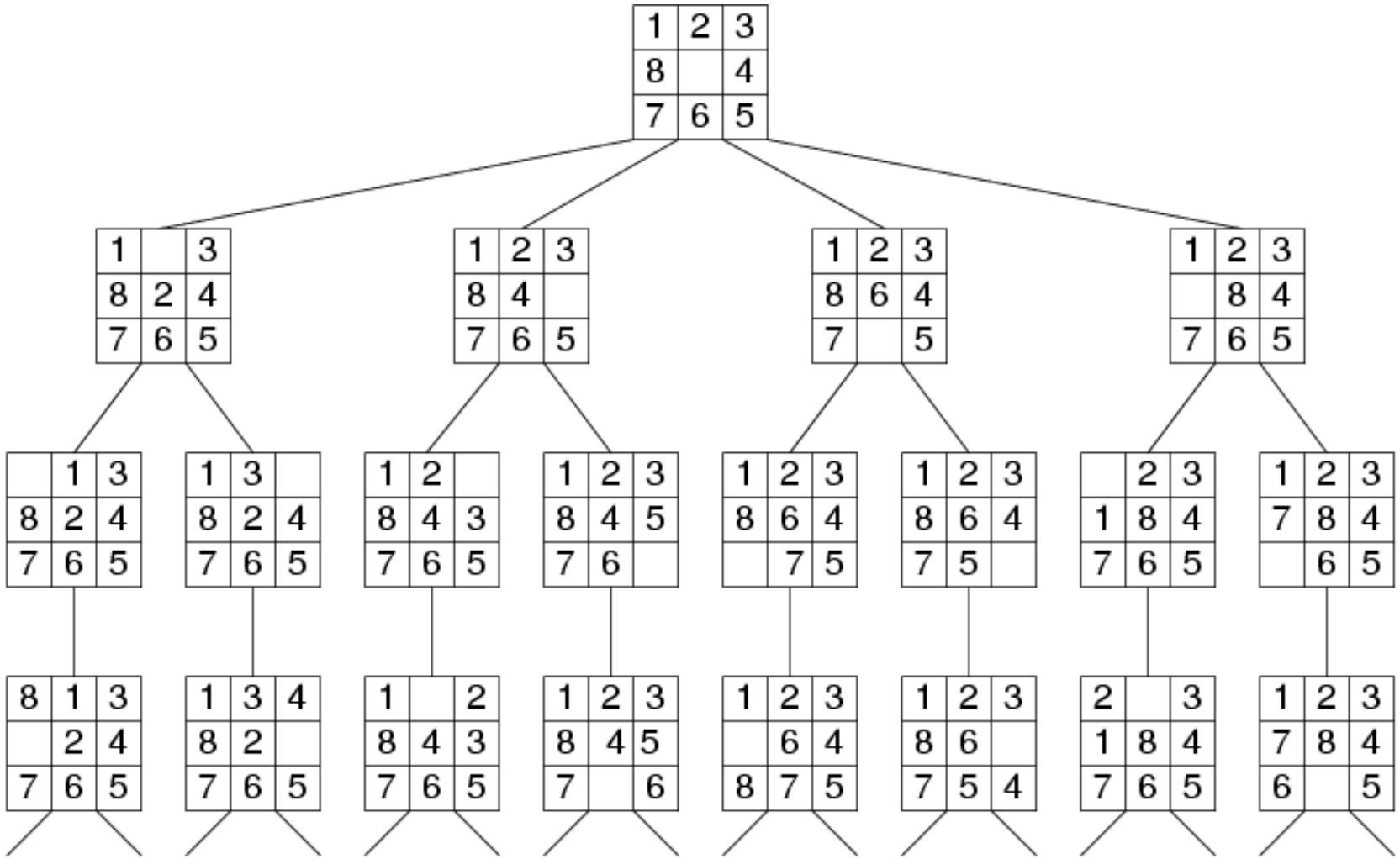


- Solution: ***sb-down, sb-left, sb-up,sb-right, sb-down***
- *Path cost**: 5** steps to reach the goal*

State space of the 8-puzzle generated by “move blank” operations

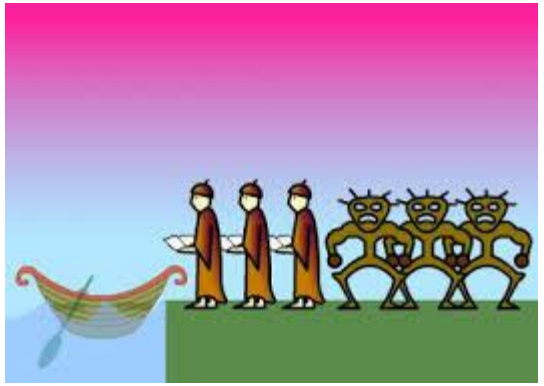


Fragment of 8-Puzzle Problem Space



Problem Formulation: Missionaries and Cannibals

- Three missionaries and three cannibals are on the left bank of a river.
- There is one boat which can hold one or two people.
- Find a way to get everyone to the right bank, without ever leaving a group of missionaries in one place outnumbered by cannibals in that place.



Missionaries and cannibals

- States: three numbers (i, j, k) representing the number of missionaries, cannibals, and boats on the left bank of the river.
- Initial state: (3, 3, 1)
- Operators: take one missionary, one cannibal, two missionaries, two cannibals, one missionary and one cannibal across the river in a given direction
- Goal Test: reached state (0, 0, 0)?
- Path Cost: Number of crossings.

Formalization of the M&C Problem

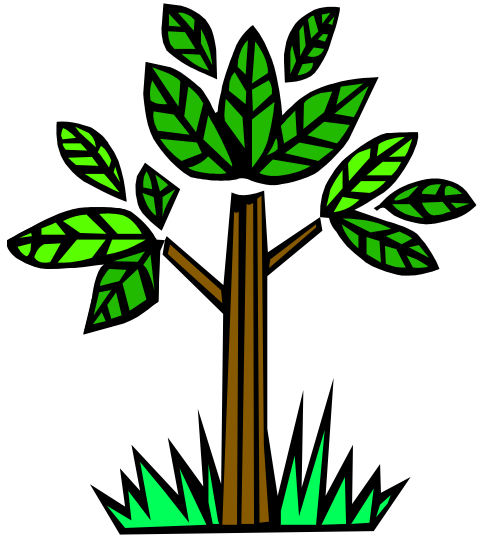
- **State space**: triple (x,y,z), where x,y, and z represent the number of missionaries, cannibals and boats currently on the original bank.
- **Initial State**: (3,3,1)
- **Successor function**: From each state, either bring one missionary, one cannibal, two missionaries, two cannibals, or one of each type to the other bank.
- **Note**: Not all states are attainable (e.g., (0,0,1)), and some are illegal.
- **Goal State**: (0,0,0)
- **Path Costs**: 1 unit per crossing

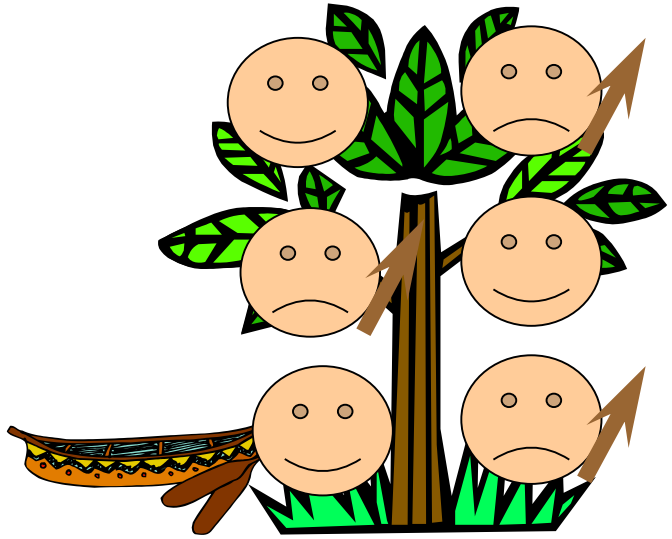
Missionaries and Cannibals

Solution = the sequence of actions within the path :

[(3,3,1)→ (2,2,0)→(3,2,1) →(3,0,0) →(3,1,1) →(1,1,0) →(2,2,1) →(0,2,0) →(0,3,1) →(0,1,0) → (0,2,1) →(0,0,0)]

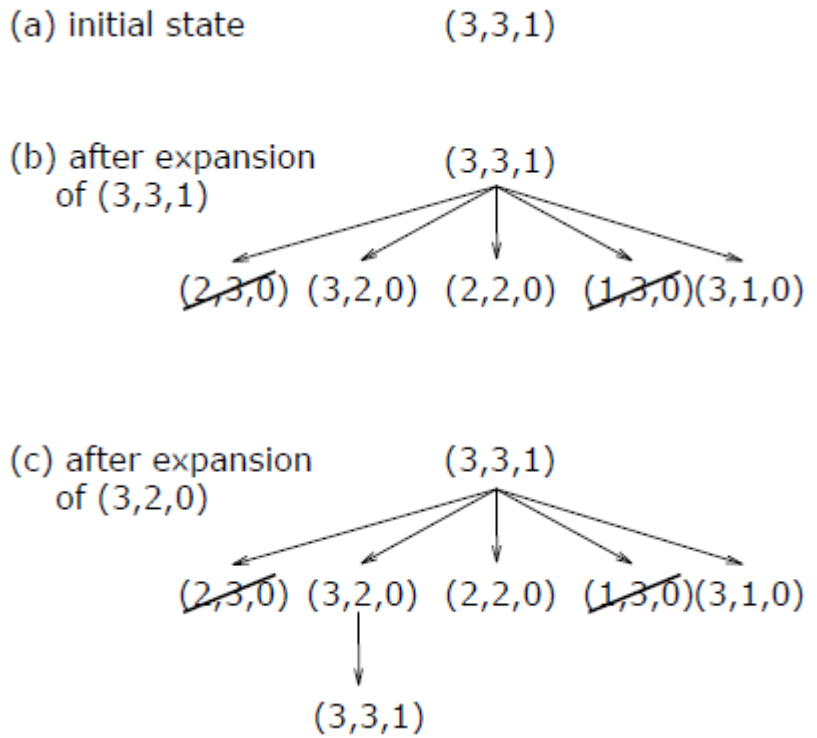
Cost = 11 crossings





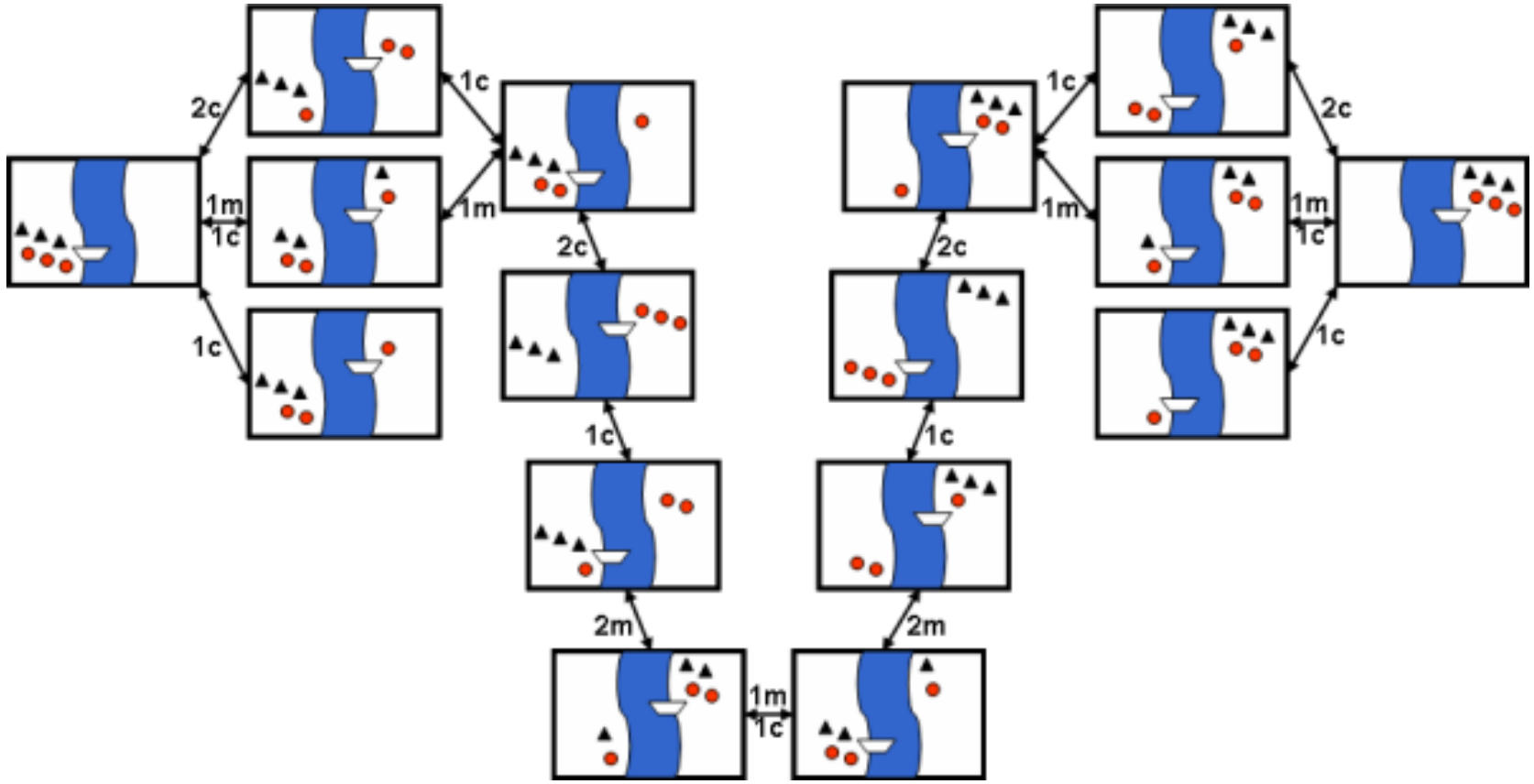
Missionaries and Cannibals

- From the initial state, produce all successive states step by step search tree.



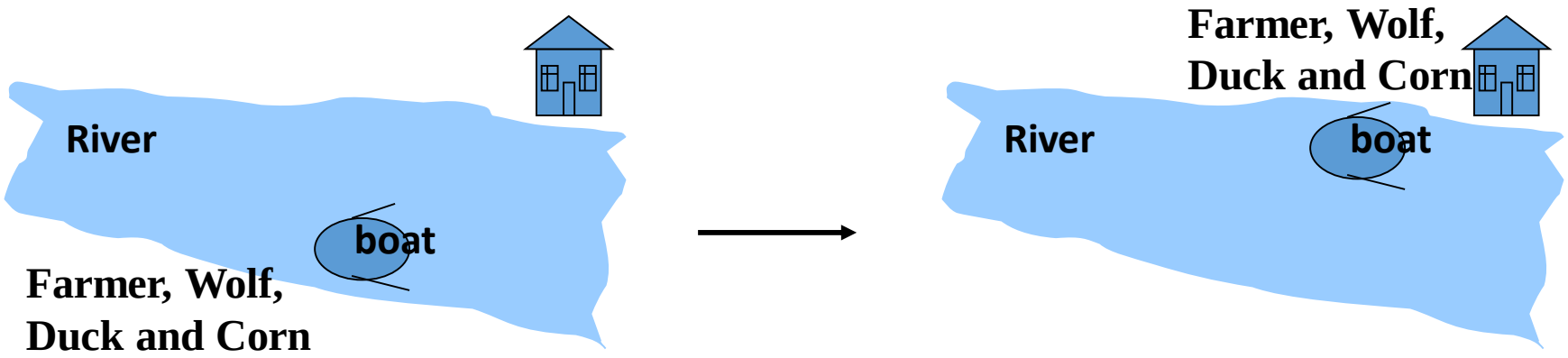
Missionaries and Cannibals

- *Search-space for the Missionaries and Cannibals problem*



The River Problem

- A farmer wishes to carry a wolf, a duck and corn across a river, from the south to the north shore. The farmer has a small rowing boat. The boat can only carry at most the farmer and one other item.
- If left unattended the wolf will eat the duck and the duck will eat the corn.

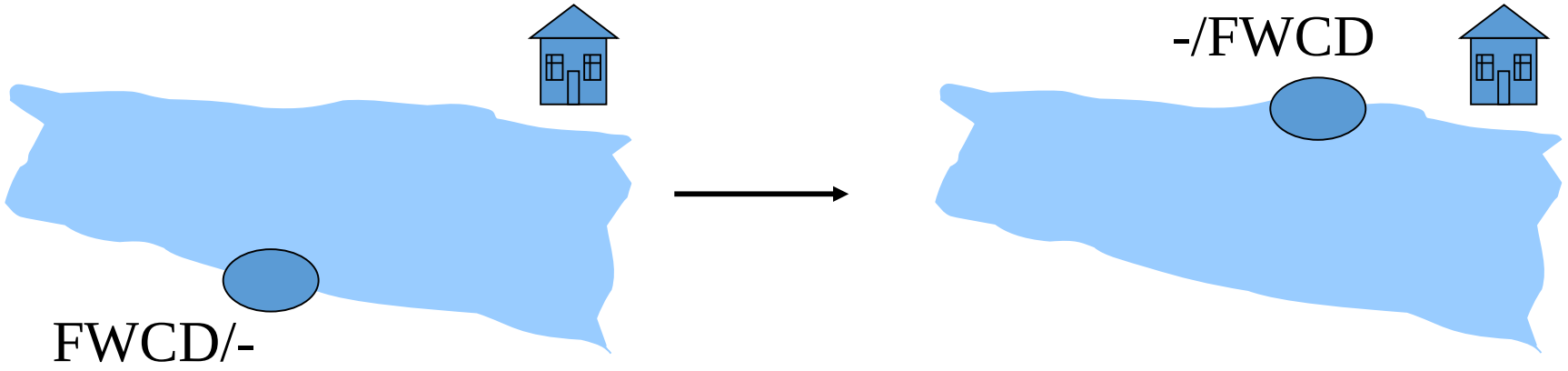


How can the farmer safely transport the wolf, the duck and the corn to the opposite shore?

The River Problem

- The River Problem:

F=Farmer W=Wolf D=Duck C=Corn /=River



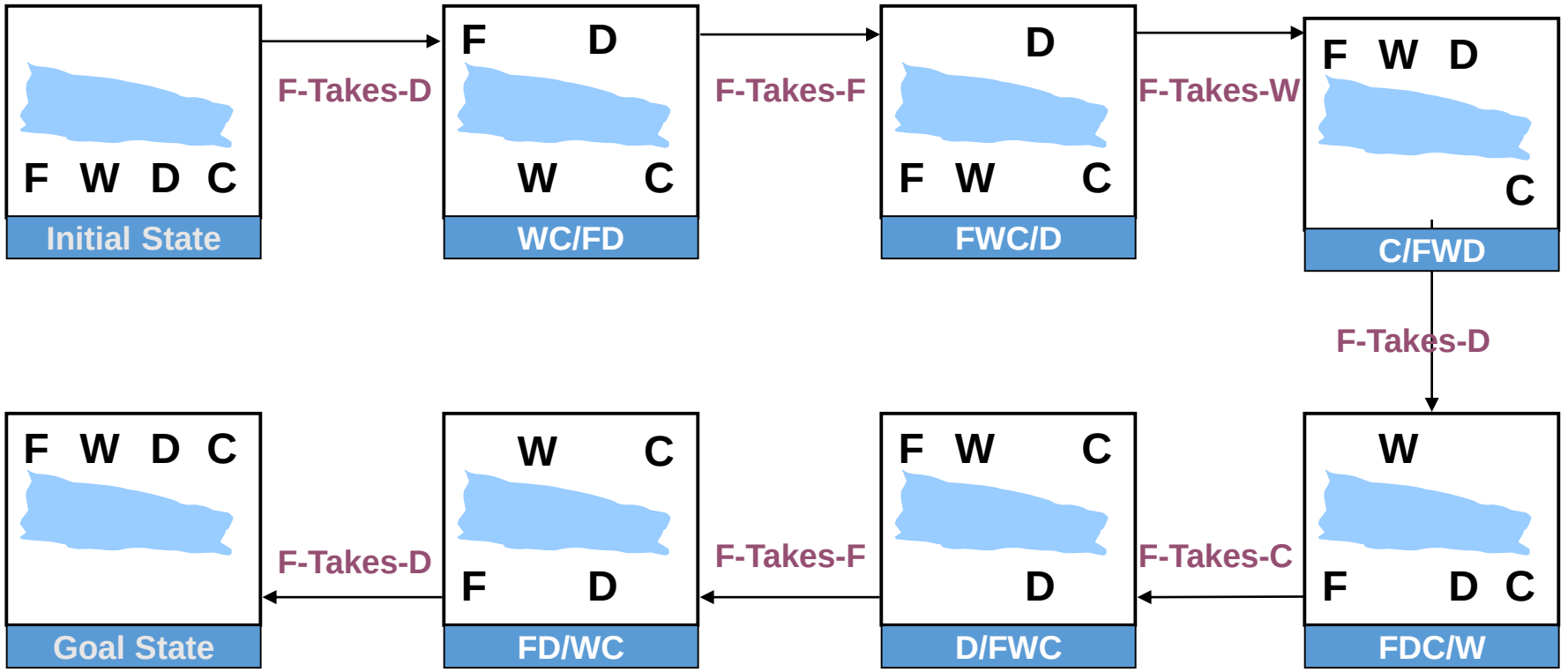
How can the farmer safely transport the wolf, the duck and the corn to the opposite shore?

The River Problem

- Problem formulation:
  - **State representation**: location of farmer and items in both sides of river
    [items in South shore / items in North shore] : (FWDC/-, FD/WC, C/FWD …)
  - **Initial State**: farmer, wolf, duck and corn in the south shore FWDC/-
  - **Goal State**: farmer, duck and corn in the north shore -/FWDC
  - **Operators**: the farmer takes in the boat at most one item from one side to the other side
    (F-Takes-W, F-Takes-D, F-Takes-C, F-Takes-Self [himself only])
  - **Path cost**: the number of crossings

The River Problem

- **Problem solution**: (path Cost = 7)
  - While there are other possibilities here is one **7** step solution to the river problem



Summary

- **Search**: process of constructing sequences of actions that achieve a goal given a problem.
- It is assumed that the environment is **observable, deterministic, static and completely known**.
- **Goal formulation** is the first step in solving problems by searching. It facilitates problem formulation.
- **Formulating a problem** requires specifying five components:
  - State representation,
  - Initial state,
  - Goal state,
  - Operators (actions), and
  - Path cost function.